

Manual Técnico

AppTurnos / SWALP

Version 1.0 | 09/08/2026

Contenido

Para ver el índice: clic derecho sobre el area de abajo > Actualizar campos.

Contenido	2
Manual Técnico — AppTurnos / SWALP	5
Índice	5
1. Visión general	6
1.1 Qué es el sistema y qué problema resuelve.....	6
1.2 Stack y versiones exactas	6
1.3 Madurez del proyecto y estado actual	7
2. Puesta en marcha	7
2.1 Requisitos previos.....	7
2.2 Entorno virtual e instalación	8
2.3 Variables de entorno	8
2.4 Base de datos y migraciones	8
2.5 Datos de arranque	9
2.6 Arrancar y comprobar que funciona	9
2.7 Docker / docker-compose	10
3. Arquitectura	10
3.1 Diagrama de contexto	10
3.2 Capas y responsabilidad de cada una.....	11
3.3 Flujo de una petición de punta a punta	13
3.4 Patrones aplicados y por qué	14
3.5 Qué NO se hizo y por qué.....	15
4. Estructura del proyecto	16
4.1 Árbol de carpetas	16
4.2 Apps Django.....	16
4.3 Convenciones de nombres y de ubicación	17
5. Modelos y base de datos.....	18
5.1 Diagrama entidad-relación	18
5.2 Ficha por modelo.....	19
5.3 Migraciones	26

5.4 Consultas críticas y su coste	27
6. Endpoints y URLs	28
6.1 Tabla de rutas	28
6.2 Fichas de endpoints no triviales	31
7. Servicios y lógica de dominio	35
7.1 Mapa de servicios	35
7.2 Validadores	38
7.3 Tareas programadas, outbox y comandos de gestión.....	39
8. Reglas de negocio	40
8.1 Estados de una solicitud	40
8.2 Principios transversales	40
8.3 Reglas por área	43
9. Autenticación, autorización y seguridad	45
9.1 Modelo de usuario y roles	45
9.2 Permisos: dónde se comprueban	46
9.3 Sesiones, CSRF, CSP y cabeceras	49
9.4 Datos sensibles y qué nunca se registra.....	50
9.5 Páginas de error y trazabilidad de la petición	50
10. Configuración	57
Diferencias dev / test / producción.....	60
11. Dependencias	61
11.1 Producción (requirements.txt)	61
11.2 Desarrollo (requirements-dev.txt)	62
11.3 Frontend y externas	62
12. Recetas (how-to)	63
12.1 Añadir un tipo de solicitud	63
12.2 Añadir un campo a un modelo	64
12.3 Correr los tests	64
12.4 Depurar un fallo de validación	64
12.5 Regenerar la documentación	64
12.6 Desplegar	65
13. Despliegue	65

13.1 Topología	65
13.2 Pipeline y pasos	66
13.3 Checklist previo	66
13.4 Rollback	67
13.5 Observabilidad.....	67
13.6 Alarma obligatoria: el cierre semanal falla ABIERTO.....	69
14. Pruebas.....	71
14.1 Cómo se corren	71
14.2 Cómo está partida la suite.....	72
14.3 Base de datos de test	72
14.4 Cobertura y áreas sin cubrir	72
15. Decisiones técnicas (ADR)	73
16. Zonas frágiles y bugs conocidos	74
16.1 El catálogo de protecciones.....	74
16.2 Zonas que se rompen fácil.....	75
16.3 Bugs y riesgos abiertos	76
16.4 Resueltos que conviene recordar	78
17. Cambios técnicos relevantes	80
Páginas de error propias y trazabilidad de la petición	80
Seguridad de los enlaces de aprobación por correo	81
Temporada y cambio de descanso (rama fix/cambio-descanso-temporada).....	82
Rendimiento y consistencia.....	83
Vocabularios y constantes.....	83
Limpieza de código muerto	84
Calendario y calidad	84
18. Por confirmar.....	84

Manual Técnico — AppTurnos / SWALP

Documentación para el desarrollador que llega mañana y tiene que instalar el proyecto, entenderlo, tocarlo sin romperlo y desplegarlo.

Toda afirmación técnica lleva su origen entre paréntesis, en formato `ruta/archivo:línea` relativo a `AppTurnosExplora/` salvo que se indique otra raíz. Las líneas se verificaron contra el código el **2026-08-08**. Lo que no se pudo verificar está en [§ 18 Por confirmar](#).

Este documento **no reemplaza** la documentación de `docs/`: la ordena y la enlaza. Cuando un tema ya está cubierto en profundidad, aquí se resume y se remite.

La terminología de dominio (explorador, jornada, doblada, cesión, pago, temporada, alternancia, D FDS...) es la misma que en [manual_usuario.md](#) (`./manual_usuario.md`) § 1.3; ese glosario es la referencia común y no se duplica aquí.

Índice

1. [Visión general](#)
2. [Puesta en marcha](#)
3. [Arquitectura](#)
4. [Estructura del proyecto](#)
5. [Modelos y base de datos](#)
6. [Endpoints y URLs](#)
7. [Servicios y lógica de dominio](#)
8. [Reglas de negocio](#)
9. [Autenticación, autorización y seguridad](#)
10. [Configuración](#)
11. [Dependencias](#)
12. [Recetas \(how-to\)](#) (how-to)](#12-recetas-how-to)
13. [Despliegue](#)
14. [Pruebas](#)
15. [Decisiones técnicas \(ADR\)](#) (ADR)](#15-decisiones-técnicas-adr)
16. [Zonas frágiles y bugs conocidos](#)
17. [Cambios técnicos relevantes](#)
18. [Por confirmar](#)

1. Visión general

1.1 Qué es el sistema y qué problema resuelve

AppTurnos (SWALP) programa los turnos de un equipo de **exploradores** del Parque Explora y gestiona los acuerdos con los que intercambian jornadas entre ellos. Cada persona tiene una jornada base (AM o PM) y una programación de descansos que depende del calendario — festivos, mantenimiento, temporada— y de la alternancia publicada de fines de semana. Sobre esa base, el sistema permite seis tipos de acuerdo entre compañeros, cada uno con reglas propias.

El núcleo técnico no es el calendario: es el **motor de solicitudes**. Seis formularios que modifican turnos propios y *de terceros*, generan deuda de jornada, exigen doble aprobación (compañero + supervisor) y se pueden revertir dentro de una ventana de 30 minutos. Casi toda la complejidad del repositorio está ahí: validar que un acuerdo es legal contra el estado *real* del día, materializarlo en filas de **Turno**, poder deshacerlo exactamente, e impedir que dos cambios concurrentes se pisen. Fuera del sistema quedan la nómina y el fichaje real: AppTurnos programa y contabiliza deuda, no asistencia.

1.2 Stack y versiones exactas

Componente	Versión / elección	Evidencia
Python	3.12	Dockerfile
Django	5.2.16 — sin Django REST Framework	requirements.txt:4
Base de datos	MySQL 8.0; driver PyMySQL 1.1.2 instalado como MySQLdb si falta el nativo	config/settings.py:115-119, requirements.txt:14
Servidor de aplicación	Gunicorn 23.0.0, 3 workers	requirements.txt:11, docker-compose.hostdb.yml
Estáticos	WhiteNoise 6.8.2 (sin Nginx en el contenedor)	config/settings.py:73
Frontend	AdminLTE 3.2 + JavaScript vanilla, sin paso de build	docs/03-arquitectura/TECNOLOGIAS_FRONTEND.md
Historial	django-simple-history 3.8.0	config/settings.py:53
Seguridad	django-axes 7.0.1, django-csp 4.0, django-cors-headers 4.9.0	config/settings.py:51-52, 74

Configuración	django-environ 0.14.0	config/settings.py:20-24
Caché	LocMem / Redis / tabla MySQL, según CACHE_URL	config/settings.py:283-316
Exportación	openpyxl 3.1.5	requirements.txt:12
Zona horaria	America/Bogota, USE_TZ=True, idioma es-co	config/settings.py:209-212

No hay API REST pública: los endpoints JSON existen para el propio frontend.

1.3 Madurez del proyecto y estado actual

Proyecto en **pre-producción**. Lo que eso significa en la práctica:

- El esquema está maduro: 35 archivos de migración en `solicitudes/migrations/`, más los de `turnos`, `empleados` y `permisos` (recuento exacto de estos tres pendiente, § 18).
- La suite de pruebas es amplia: 43 archivos en `solicitudes/tests/` y 33 repartidos entre `turnos`, `empleados`, `permisos`, `core` e `integration_tests`.
- El despliegue en AWS está documentado y hay imagen Docker, pero la infraestructura definitiva

sigue con pendientes ([ADR 005](#) (./03-arquitectura/adr/005-pendientes-aws.md)).

- La política CSP estricta está publicada en modo **report-only**, a la espera de confirmar que no genera violaciones (`config/settings.py:399-418`).
- Los datos de la base de desarrollo son descartables: producción arranca limpia. No hay backfills ni migraciones de datos planificadas.

2. Puesta en marcha

Tutorial reproducible desde cero. Asume que no conoces el proyecto.

2.1 Requisitos previos

Requisito	Versión	Por qué
Python	3.12	Es la versión de la imagen (<code>Dockerfile</code>)

MySQL	8.0	Motor de DATABASES (config/settings.py:164)
Git	cualquiera	—
Microsoft Word	opcional	Solo para regenerar los PDF de documentación (usa COM)

Trampa de orientación nº 1: la raíz del proyecto Django es `AppTurnosExplora/` (ahí está `manage.py`), no la raíz del repositorio.

Trampa nº 2: el `docker-compose.yml` de la raíz del repositorio levanta **SonarQube**, no la aplicación. Los de la aplicación son `AppTurnosExplora/docker-compose.local.yml` y `AppTurnosExplora/docker-compose.hostdb.yml`.

2.2 Entorno virtual e instalación

```
cd AppTurnosExplora
python -m venv .env
.venv\Scripts\activate          # Windows
pip install -r requirements-dev.txt # incluye requirements.txt con -r
```

`requirements-dev.txt` hereda producción con `-r requirements.txt` y añade solo herramientas de desarrollo (`requirements-dev.txt:3`).

2.3 Variables de entorno

```
copy .env.example .env      # y edita los valores
```

`.env.example` es la plantilla completa. Las variables **obligatorias** sin valor por defecto son `SECRET_KEY`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD` y `DEFAULT_FROM_EMAIL`: si falta cualquiera, `django-environ` lanza `ImproperlyConfigured` al importar `settings` y el proceso no arranca (`config/settings.py:32,165-167,241-243`). La tabla completa está en [§ 10 Configuración](#).

`ENVIRONMENT` (`development` | `production`) gobierna todo el archivo de settings, que es único: no hay `settings/local.py` (`config/settings.py:26-27`).

2.4 Base de datos y migraciones

```
CREATE DATABASE bdturnosex CHARACTER SET utf8mb4;
```

```
python manage.py migrate
```

Si usas caché en tabla (`CACHE_URL=db://cache_appturnos`), hace falta además, una sola vez:

```
python manage.py createcachetable
```

(`config/settings.py:276-278`).

Trampa conocida en MySQL local: las tablas de zonas horarias vienen vacías y el admin de Django falla al filtrar por fecha. RDS/Aurora ya vienen pobladas; en local hay que cargarlas con `mysql_tzinfo_to_sql`.

2.5 Datos de arranque

```
python manage.py createsuperuser
python manage.py instalar_calendario_colombiano      # festivos
python manage.py actualizar_codigos_estrategia      # sincroniza codigo_estrategia
```

Además hacen falta datos que **no** se generan solos:

- Las dos filas de **Jornada** con nombre exacto **AM** y **PM**. El motor las busca por nombre literal y no se pueden borrar (`empleados/models.py:19-25,39-41`).
- Los roles **Supervisor** y **Explorador**, también por nombre exacto (`empleados/models.py:82-85`).
- Los seis **TipoSolicitudCambio** con los nombres de `core.constants.TipoSolicitud`

(`core/constants.py: CAMBIO TURNO, CAMBIO DESCANSO, CT PERMANENTE, DOBLADA, DOBLADA PERMANENTE, D FDS`).

- El calendario anual: temporada, festivos, mantenimiento, alternancia de fines de semana y descansos de semana. **Se carga a mano cada diciembre**; que un año futuro esté vacío es intencional, no un fallo. Procedimiento en [04-guias/mantenimiento-anual/](#) (`./04-guias/mantenimiento-anual/`).

2.6 Arrancar y comprobar que funciona

```
python manage.py runserver
```

Comprobación	Cómo	Qué debe pasar
Proceso vivo	<code>GET /health/</code>	<code>{"status": "ok"}</code> , sin tocar la base (<code>core/health.py</code>)
Base accesible	<code>GET /health/ready/</code>	Estado que sí verifica la conexión (<code>core/health.py</code>)
Login	<code>GET /</code>	Formulario de inicio de sesión (<code>core/login/urls.py:5</code>)
Panel	<code>GET /dashboard/</code>	Portada tras autenticarse (<code>core/dashboard/urls.py:5</code>)

Configuración	<code>python manage.py check - -deploy</code>	Sin errores críticos
---------------	---	----------------------

Ninguno de los dos endpoints de salud requiere autenticación ni devuelve información del sistema: solo un estado (`core/health.py`, docstring del módulo).

2.7 Docker / docker-compose

Hay dos ficheros, para dos escenarios distintos:

Fichero	Qué levanta	Cuándo usarlo
<code>docker-compose.local.yml</code>	MySQL 8.0 efímero (puerto host 3307) + la app con la imagen de producción	Probar la imagen real sin tocar tu base local
<code>docker-compose.hostdb.yml</code>	Solo la app, apuntando a tu MySQL del host vía <code>host.docker.internal</code>	Probar la imagen contra tus datos reales

```
docker compose -f docker-compose.local.yml up --build -d # http://localhost:8000
docker compose -f docker-compose.local.yml down -v
```

Tres detalles que evitan sorpresas:

19. Ambos ficheros fijan `ENVIRONMENT=production` pero `SECURE_HTTPS=False`, para poder navegar en

HTTP local sin caer en el redirect a HTTPS (`docker-compose.local.yml`, sección `environment`).

20. `docker-compose.hostdb.yml` **sobrescribe el `command`** para arrancar solo Gunicorn, sin `migrate`: no toca el esquema de tu base real (`docker-compose.hostdb.yml`, clave `command`).

21. El puerto de MySQL en `local` se publica en **3307** para no chocar con tu MySQL local en 3306.

Guía completa: [MANUAL DOCKER LOCAL.md](#) (./05-referencia/deployment/MANUAL_DOCKER_LOCAL.md).

3. Arquitectura

3.1 Diagrama de contexto

```
flowchart TB
    EXP["Explorador<br/>(navegador)"]
    SUP["Supervisor / staff"]
    subgraph APP ["AppTurnos - Django 5.2 monolito"]
```

```

WEB["Vistas HTML<br/>AdminLTE + JS vanilla"]
API["Endpoints JSON<br/>(consumo propio)"]
ORQ["Motor de solicitudes<br/>orchestrator + factory + strategies"]
TUR["Motor de turnos<br/>turnos/services"]
end
DB["MySQL 8.0<br/>turnos, solicitudes, deudas"]
CACHE["Caché<br/>LocMem / Redis / tabla"]
OUTBOX[["EmailOutbox<br/>(tabla)"]]
SMTP["SMTP / SES"]
CRON["cron:<br/>procesar_email_outbox"]

EXP --> WEB
SUP --> WEB
WEB --> API
WEB --> ORQ
ORQ --> TUR
ORQ --> DB
TUR --> DB
TUR --> CACHE
ORQ --> OUTBOX
CRON --> OUTBOX
OUTBOX --> SMTP
SMTP -->|enlace firmado de aprobación| EXP

```

Qué muestra. AppTurnos es un monolito Django único, sin servicios separados. Exploradores y supervisores usan el mismo navegador y las mismas vistas; lo que cambia es el permiso. Todo lo que modifica turnos pasa por el motor de solicitudes, que escribe en MySQL y encola correos en la tabla `EmailOutbox` dentro de la misma transacción. El envío real lo hace después un cron que ejecuta `procesar_email_outbox` (`solicitudes/models.py:44-64`). Los correos llevan enlaces firmados que permiten aprobar sin iniciar sesión: es la única entrada al sistema que no pasa por el login. La caché guarda el estado de "Mis Turnos" y se invalida al aprobar o cancelar.

3.2 Capas y responsabilidad de cada una

Capa	Dónde vive	Responsabilidad	Qué NO hace
HTTP	<code>*/views/</code>	Parsear la petición, comprobar sesión y permisos, devolver JSON o HTML	No contiene reglas de negocio
Orquestación	<code>solicitudes/services/solicitud_orchestrator.py</code>	Encadena los chequeos transversales y despacha al tipo	No conoce las reglas de cada tipo
Casos de	<code>solicitudes/use_cases/</code>	Nombra la intención: crear,	No habla

uso		aprobar, cancelar	HTTP
Dominio	<code>solicitudes/domain/</code>	Máquina de estados, bloqueos, invariantes	No consulta el ORM salvo lo imprescindible
Estrategias	<code>solicitudes/services/strategies/</code>	Reglas propias de cada uno de los seis tipos	No sabe qué otros tipos existen
Validadores	<code>solicitudes/services/validators/</code>	Reglas reutilizables entre tipos	No escribe
Servicios de aplicación	<code>solicitudes/services/*_aplicacion_service.py</code>	Materializa y revierte turnos y deudas	No valida
Repositorios	<code>solicitudes/repositories/</code>	Consultas al ORM aisladas	No decide
Modelos	<code>*/models.py</code>	Persistencia + invariantes estructurales (constraints)	—

La descripción larga está en [03-arquitectura/ARQUITECTURA.md](#) (./03-arquitectura/ARQUITECTURA.md) y la justificación de la capa de servicios en [ADR 001](#) (./03-arquitectura/adr/001-service-layer-y-orchestrator.md).

Middleware. El orden de `MIDDLEWARE` (`config/settings.py:70-85`) no es decorativo; tres posiciones están fijadas por una razón concreta:

Posición	Middleware	Por qué ahí
1.º	<code>core.errors.RequestIDMiddleware</code> (<code>config/settings.py:71</code>)	Etiqueta la petición con su identificador. Lo que quede por encima no queda etiquetado, así que va arriba del todo (§ 9.5)
2.º	<code>corsheaders.middleware.CorsMiddleware</code>	Debe responder al preflight antes que el resto

último – 1	<code>axes.middleware.AxesMiddleware</code>	La propia librería exige ir al final
último	<code>core.middleware.AperturaAnioMiddleware</code>	Bloquea al supervisor si falta planificar el año; necesita sesión y usuario ya resueltos

Fuera de producción, `debug_toolbar.middleware.DebugToolbarMiddleware` se inserta en la posición 0 (`config/settings.py:87-88`), por delante incluso del identificador.

3.3 Flujo de una petición de punta a punta

```
sequenceDiagram
    autonumber
    actor E as Explorador
    participant V as ProcesarSolicitudView
    participant U as CrearSolicitudUseCase
    participant O as SolicitudOrchestrator
    participant F as SolicitudFactory
    participant S as Strategy del tipo
    participant DB as MySQL
    actor C as Compañero
    actor SUP as Supervisor
    participant AP as Servicio de aplicación

    E->>V: POST /solicitudes/procesar-solicitud/
    V->>V: tipo_solicitud_id presente y existe
    V->>U: execute(POST, tipo, empleado)
    U->>O: procesar(...)
    O->>O: dedupe del POST idéntico (10 s)
    O->>O: sanción del solicitante
    O->>O: cierre semanal sobre las fechas objetivo
    O->>O: sanción del compañero
    O->>O: restricción médica (advierde, no bloquea)
    O->>F: validar_solicitud(tipo, datos)
    F->>S: validar_solicitud(datos)
    S-->>F: (ok, mensaje)
    F->>S: crear_solicitud(datos)
    S->>DB: SolicitudCambio + detalle (estado=pendiente)
    S->>DB: EmailOutbox (misma transacción)
    O-->>E: 201 {success, solicitud_id}

    C->>DB: aprueba como receptor
    SUP->>DB: aprueba como supervisor
    Note over F,AP: al completarse las dos aprobaciones
    F->>S: revalidar_para_aprobar(solicitud)
    F->>AP: aplicar_cambios(solicitud)
    AP->>DB: snapshot previo + turnos nuevos + deudas
    AP->>DB: estado = aprobada, fecha_resolucion = ahora

    E->>DB: cancelar (≤ 30 min)
    Note over AP,DB: guardia LIFO + guardia de integridad
    AP->>DB: restaurar snapshot, cancelar deudas, reconciliar
    AP->>DB: estado = cancelada, fecha_cancelacion = ahora
```

Qué muestra. El ciclo de vida completo de una solicitud. Los pasos 1-11 ocurren en una sola petición POST y terminan con la solicitud **pendiente**: nada se ha aplicado todavía al calendario. La aplicación real solo ocurre cuando se completan las dos aprobaciones, y va precedida de una revalidación contra el estado actual, porque entre la creación y la aprobación el mundo pudo cambiar. La cancelación deshace exactamente lo aplicado usando el snapshot capturado antes de aplicar, y solo si las dos guardias lo permiten.

3.4 Patrones aplicados y por qué

Strategy — un archivo por tipo de solicitud. Los seis tipos comparten la mayor parte del flujo pero difieren en reglas casi incompatibles: una doblada genera deuda, un cambio de descanso no; un CT permanente opera sobre un rango, un CT sencillo sobre un día. Resuelto con condicionales habría producido una vista de miles de líneas donde tocar un tipo rompe otro. `SolicitudStrategy` (`solicitudes/services/strategies/base_strategy.py`) define el contrato —`validar_solicitud`, `crear_solicitud`, `aplicar_cambios`, `revalidar_para_aprobar`, `get_empleados_disponibles`— y cada tipo lo implementa. El orquestador no sabe con qué tipo trabaja.

Factory con búsqueda en cuatro niveles. Los tipos viven en base de datos (`TipoSolicitudCambio`) y sus nombres los escriben personas. `SolicitudFactory.get_strategy` (`solicitudes/services/solicitud_factory.py:123`) busca en cascada: campo `codigo_estrategia` normalizado (:143), `codigo_estrategia` directo (:154), nombre directo (:176) y, si nada casa, estrategia por defecto con `warning` en el log (`solicitudes/services/solicitud_factory.py:182-187`).

Validadores reutilizables. Reglas como "no en día de mantenimiento" o "no contigo mismo" aplican a casi todos los tipos. `SolicitudValidator` (`solicitudes/services/validators/base_validator.py`) las concentra y los validadores específicos —`ct_validator.py`, `ct_permanente_validator.py`, `doblada_validator.py`— añaden lo suyo.

Máquina de estados propia, sin librería. Un único mapa de transiciones legales (`solicitudes/domain/estado_machine.py:11-18`) y una función que las valida (:28, lanza `EstadoTransicionError` en :47-53). El motivo de no usar `django-fsm` está en [ADR 002](#) (./03-arquitectura/adr/002-fsm-sin-libreria-externa.md).

Casos de uso. `use_cases/` nombra la intención por encima de la implementación: `CrearSolicitudUseCase`, `AprobarComoReceptorUseCase`, `AprobarComoSupervisorUseCase`, `AprobarAmbosRolesUseCase` (`solicitudes/use_cases/aprobar_solicitud.py:39`), `CancelarSolicitudUseCase`.

Repositorios. `solicitudes/repositories/` aísla las consultas del ORM para que las estrategias pidan datos sin escribir queriesets.

Outbox de correos. La fila de correo se escribe **dentro** de la misma transacción que el cambio de negocio, con clave de idempotencia única; el envío real ocurre después y se reintenta si falla

([solicitudes/models.py:44-64,77,93](#)). Sin esto, un proceso que muriera entre el COMMIT y el envío perdía el correo en silencio.

Todo el catálogo de protecciones —bloqueo pesimista, snapshot-once, deuda idempotente, guardias que fallan cerrado, invariantes en la base de datos— vive en ``PROTECTION_PATTERNS.md`` (raíz del repositorio, 37 patrones). Se resume en [§ 16](#).

3.5 Qué NO se hizo y por qué

Alternativa descartada	Motivo	Dónde consta
Django REST Framework	No hay consumidores externos; los endpoints JSON los usa el propio frontend	Ausente de <code>INSTALLED_APPS</code> (config/settings.py:43-62)
<code>django-fsm</code>	Las transiciones son pocas y fijas; una librería añadía dependencia sin cerrar el hueco real (asignar <code>estado</code> a mano)	ADR 002 (<code>./03-arquitectura/adr/002-fsm-sin-libreria-externa.md</code>)
Celery / broker	El único trabajo diferido son los correos; se resolvió con tabla outbox + cron	solicitudes/models.py:44-64
<code>django-redis</code>	El backend Redis nativo de Django 5 basta; solo hace falta el cliente <code>redis</code>	config/settings.py:280-281
Paso de build en frontend	JS vanilla y AdminLTE servidos como estáticos; no hay bundler	docs/03-arquitectura/TECNOLOGIAS_FRONTEND.md
Índice único parcial en MySQL	MySQL no soporta <code>UniqueConstraint(condition=...)</code> ; se usa columna discriminante nullable	turnos/models.py:55-60,80-83
Settings partidos por entorno	Un archivo único gobernado por <code>ENVIRONMENT</code>	config/settings.py:26-27

4. Estructura del proyecto

4.1 Árbol de carpetas

C:\appTurnos\	raíz del repositorio
├─ AppTurnosExplora\	← raíz del proyecto Django (manage.py)
│ ├─ config\	settings.py, urls.py, wsgi.py, db.py, paths.py
│ └─ core\	login, dashboard, mixins, constantes, health,
cache	
│ ├─ empleados\	personas, roles, salas, competencias,
restricciones, sanciones	
│ └─ turnos\	turnos, jornadas, días especiales, descansos,
alternancia	
│ ├─ solicitudes\	el dominio central: los seis tipos de solicitud
│ ├─ permisos\	PDH y permisos especiales
│ ├─ templates\	plantillas HTML (incluye emails\)
│ ├─ static\	JS, CSS, AdminLTE, plugins autohospedados
│ ├─ integration_tests\	pruebas de integración
│ ├─ scripts\	utilidades manuales (NO recogidas por pytest)
│ ├─ docs\	esta documentación
│ ├─ Dockerfile	
│ ├─ docker-compose.local.yml	app + MySQL efímero
│ └─ docker-compose.hostdb.yml	app contra tu MySQL del host
└─ PROTECTION_PATTERNS.md	38 patrones de concurrencia, idempotencia y
seguridad	
├─ instructivos\	fuentes de negocio en .docx / .mwb
└─ docker-compose.yml	⚠ solo SonarQube, NO la aplicación

4.2 Apps Django

App	Responsabilidad	Contiene	Depende de
core.login	Autenticación	LoginFormView, LogoutUserView (core/login/urls.py:5-6)	empleados
core.dashboard	Portada	DashboardView (core/dashboard/urls.py:5)	todas
core (no-app)	Mixins de permiso, constantes de dominio, caché, health, middleware, test runner	core/mixins.py, core/constants.py, core/health.py, core/test_runner.py	ninguna
empleados	Personas, jornadas, roles, salas, competencias, restricciones, sanciones, PDH (vistas)	empleados/models.py (303 líneas), empleados/views/	core
turnos	Turnos, días	turnos/models.py (397),	empleados

	especiales, descansos de semana, alternancia, apertura de año, reportes	<code>turnos/services/</code> (13 servicios), <code>turnos/api/</code>	
solicitudes	Dominio central: seis tipos, deudas, aprobaciones, notificaciones, cierre semanal, reprogramaciones, outbox	<code>solicitudes/models.py</code> (847), 29 servicios, 7 estrategias, 4 validadores, 18 módulos de vistas	empleados, turnos
permisos	PDH y permisos especiales	<code>permisos/models.py</code> (158)	empleados, solicitudes

Dependencias en un solo sentido: `core` → `empleados` → `turnos` → `solicitudes` → `permisos`.
`core/constants.py` no importa ninguna app, precisamente para no crear ciclos (`core/constants.py`, docstring).

4.3 Convenciones de nombres y de ubicación

Elemento	Convención	Ejemplo
Estrategia	<code>services/strategies/<tipo>_strategy.py</code> , clase <code><Tipo>Strategy</code>	<code>doblada_strategy.py</code> → <code>DobladaStrategy</code>
Validador	<code>services/validators/<tipo>_validator.py</code>	<code>doblada_validator.py</code>
Servicio de aplicación	<code>services/<tipo>_aplicacion_service.py</code> , método <code>aplicar()</code> / <code>revertir()</code>	<code>cambio_descanso_aplicacion_service.py</code>
Caso de uso	<code>use_cases/<verbo>_solicitud.py</code> , clase <code><Verbo>UseCase</code>	<code>CancelarSolicitudUseCase</code>
Vista JSON	<code>views/api_*.py</code> o <code>views/*_api.py</code> , reexportada en <code>views/__init__.py</code>	<code>views/api_fin_semana.py</code>
Respuesta JSON	Siempre <code>json_ok</code> / <code>json_error</code> de <code>core/utils/json_responses.py</code>	—
Test	<code>test_*.py</code> , clases <code>Test*</code> , funciones <code>test_*</code>	<code>pytest.ini:3-5</code>
Literal persistente	Nunca en línea: constante en <code>core/constants.py</code>	<code>TipoSolicitud.DOBLADA</code>

Las vistas de `solicitudes` están partidas en 18 módulos y reexportadas desde `solicitudes/views/__init__.py`; el diagnóstico que lo motivó está en [01-analisis/ANALISIS_VIOLACIONES_SRP.md](#) (`./01-analisis/ANALISIS_VIOLACIONES_SRP.md`).

5. Modelos y base de datos

Cuatro apps con modelos: `empleados` (8 modelos), `turnos` (7), `solicitudes` (14) y `permisos` (2). Casi todos llevan `HistoricalRecords()` de `django-simple-history`, así que tienen además su tabla `historical*` con el rastro de cambios; `Notificacion`, `EmailOutbox` y `TurnoArchivo` no lo llevan.

5.1 Diagrama entidad-relación

```
erDiagram
    User ||--|| Empleado : "OneToOne"
    Empleado ||--o{ EmpleadoRole : "tiene"
    Role ||--o{ EmpleadoRole : "PROTECT"
    Empleado ||--o{ CompetenciaEmpleado : "sabe salas"
    Sala ||--o{ CompetenciaEmpleado : ""
    Empleado ||--o{ RestriccionEmpleado : "restriccion medica"
    Empleado ||--o{ SancionEmpleado : "sancionado"
    Empleado ||--o{ AsignarJornadaExplorador : "jornada base"
    Jornada ||--o{ AsignarJornadaExplorador : "PROTECT"
    Empleado ||--o{ Turno : "trabaja"
    Jornada ||--o{ Turno : "PROTECT"
    Sala ||--o{ Turno : ""
    Jornada ||--o{ DescansoSemanaManual : "descansa"
    Jornada ||--o{ AsignacionEspecialManual : "trabaja el dia"

    Empleado ||--o{ SolicitudCambio : "solicita"
    Empleado ||--o{ SolicitudCambio : "recibe"
    TipoSolicitudCambio ||--o{ SolicitudCambio : "tipo"
    SolicitudCambio ||--o{ CambioPermanenteDetalle : "CT PERMANENTE"
    CambioPermanenteDetalle ||--o{ CambioPermanenteDia : "dias"
    SolicitudCambio ||--o{ DobladaDetalle : "DOBLADA / CAMBIO DESCANSO / D FDS"
    SolicitudCambio ||--o{ DobladaPermanenteDetalle : "DOBLADA PERMANENTE"
    SolicitudCambio ||--o{ DeudaExplorador : "genera"
    SolicitudCambio ||--o{ DeudaCorporativa : "genera"
    SolicitudCambio ||--o{ ReprogramacionDiaDoblada : "reprograma"
    SolicitudCambio ||--o{ Notificacion : "notifica"
    SolicitudCambio ||--o{ PDH : "opcional"
    Empleado ||--o{ PDH : "paga horas"
    DeudaCorporativa }o--o{ PDH : "deudas pagadas"
    PermisoEspecial }o--o{ PDH : "permisos pagados"
    Empleado ||--o{ PermisoEspecial : "pide"
```

Qué muestra. El centro de gravedad es `SolicitudCambio`: toda solicitud, sea del tipo que sea, es una fila de esa tabla más una fila de detalle según el tipo. Hay tres tablas de detalle, no seis:

`CambioPermanenteDetalle` para CT PERMANENTE, `DobladaPermanenteDetalle` para DOBLADA PERMANENTE y `DobladaDetalle` —la más cargada— para DOBLADA, CAMBIO DESCANSO y D FDS. CAMBIO TURNO no tiene detalle: le basta con `fecha_cambio_turno` y los snapshots de la propia solicitud (`solicitudes/models.py:150,186-201`). A la izquierda queda el calendario: `Turno` es el hecho materializado, mientras `Jornada`, `DescansoSemanalManual` y `AsignacionEspecialManual` son la planificación que dice quién trabaja cuándo. A la derecha, la contabilidad: `DeudaCorporativa` (30 minutos por doblada) y `PDH`, el pago que la salda.

5.2 Ficha por modelo

App empleados (empleados/models.py)

Modelo	Campos clave	Constraints e índices	Notas de negocio
Jornada	<code>nombre</code> (unique, choices AM/PM), <code>hora_inicio</code> , <code>hora_fin</code>	<code>unique</code> en <code>nombre</code> (:25)	Catálogo estructural. El motor la busca por nombre literal y deriva la "jornada contraria" de forma binaria; <code>NOMBRES_PROTEGIDOS = (AM, PM)</code> y la propiedad <code>es_protegida</code> impiden borrarlas (:19-23,38-41)
Empleado	<code>user</code> (OneToOne con <code>auth.User</code>), <code>nombre</code> , <code>apellido</code> , <code>cedula</code> (unique), <code>email</code> , <code>activo</code> , <code>supervisor</code> (FK a sí mismo, SET_NULL)	<code>empleado_activo_idx</code> , <code>empleado_superactivo_idx</code> (:58-61)	<code>supervisor</code> define a quién llega el correo de aprobación. <code>notificaciones_no_leidas_count()</code> (:66-68)
Role	<code>nombre</code> (unique)	<code>unique</code> (:87)	Catálogo estructural. <code>es_protegido</code> compara sin distinguir mayúsculas porque el permiso también lo hace: un rol "supervisor" concede acceso (:98-107). El docstring documenta la escalada de privilegios que hubo con búsqueda <code>icontains</code> (:73-81)
EmpleadoRole	<code>empleado</code> , <code>role</code>	<code>unique_together</code> (<code>empleado</code> , <code>role</code>), <code>empleado_role_com</code>	<code>role</code> es <code>on_delete=PROTECT</code> a propósito: borrar "Supervisor" arrastraba en silencio todas sus

		p_idx (:121-124)	asignaciones (:112-114)
Sala	nombre, activo	sala_activo_idx (:139-141)	—
CompetenciaEmpleado	empleado, sala	unique_together, comp_emp_sala_idx (:156-159)	Qué salas sabe atender cada persona
RestriccionEmpleado	empleado, fecha_inicio, fecha_fin (null), recomendacion, tipo_restriccion	rest_emp_fecha_idx, rest_tipo_idx (:179-182)	clean() exige fecha_fin >= fecha_inicio (:184-187). Advierte, no bloquea: ver SolicitudoOrchestrator.verificar_restriccion
SancionEmpleado	explorador, supervisor, fecha_inicio, fecha_fin, motivo, levantada_en, levantada_por (PROTECT), levantada_motivo	sanc_exp_fecha_idx, sanc_supervisor_idx (:230-233)	Invariante: una sanción no se borra, se levanta. fecha_fin es el fin planeado y no se toca; el fin real lo da levantada_en (:193-208). fecha_fin_efectiva puede quedar antes que fecha_inicio, y por eso se calcula en vez de guardarse (:249-260). levantar() es idempotente (:286-299). clean() prohíbe autosancionarse (:241-242)

App turnos (turnos/models.py)

Modelo	Campos clave	Constraints e índices	Notas de negocio
AsignarJornadaExplorador	explorador, jornada (PROTECT), fecha_inicio	jornada_explorador_fecha_idx (:19-21)	Sin fecha_fin: las jornadas son indefinidas (:13). La vigente es la de mayor fecha_inicio menor o igual a la fecha consultada
Turno	explorador, fecha, jornada (PROTECT), sala, tipo_cambio (null = turno normal), anulado,	turno_explorador_fecha_idx; UniqueConstraint(explorador, fecha, jornada, activo_key)	El hecho materializado. Soft-delete auditable vía anulado. activo_key vale 1 si

	motivo_anulacion, activo_key (derivado, editable=False)	= turno_unico_activo_por_jornada; CheckConstraint turno_tipo_cambio_valido (:70-95)	está activo y NULL si anulado, y se mantiene sola en save() (:98-104): existe porque MySQL no soporta índices únicos parciales. El manager por defecto TurnoActivoManager excluye anulados; Turno.all_objects los incluye (:27-36,63-65)
DiaEspecial	fecha, tipo (festivo/mantenimiento/temporada), recurrente, activo, año_planificacion, es_temporada, mes	dia_esp_anio_mes_tem_p_idx, dia_esp_fecha_tipo_activo_idx; UniqueConstraint(fecha, tipo) (:140-149)	mes, año_planificacion y es_temporada son derivados y se recalculan en cada save() para que no puedan contradecir a fecha/tipo (:152-164). La temporada manda sobre el mantenimiento: es_mantenimiento_efectivo devuelve False si el día cae en temporada (:186-200). ANIO_MIN=2000, ANIO_MAX=2100 como fuente única de los límites (:110-113)
TurnoArchivo	Espejo de Turno + fecha_archivado, turno_original_id	turno_arch_exp_fecha_idx, turno_arch_fecha_idx (:227-230)	Sin CheckConstraint a propósito: solo recibe copias ya validadas (:218-219)
DescansoSemanal	fecha (lun-vie), jornada (PROTECT), motivo, activo	UniqueConstraint(fecha, jornada), descsem_fecha_activo_idx (:268-273)	En semanas de temporada o festivo el descanso deja de ser el lunes de mantenimiento y el supervisor lo fija aquí,

			por jornada (:240-247). <code>clean()</code> rechaza sábado y domingo (:275-278)
AsignacionEspecialManual	fecha, jornada_trabaja (PROTECT), tipo (finde/festivo), activo	UniqueConstraint(fecha), asigesp_fecha_activo_idx (:318-323)	Fuente de verdad de la alternancia. Una fecha sin fila significa "sin planificar" y así se reporta; nunca se inventa un grupo (:284-296). <code>clean()</code> valida que el tipo case con el día de la semana (:325-333)
AperturaAnioConfig	inicio_recordatorio_dia/mes, inicio_bloqueo_dia/mes, bloqueo_duro	— (singleton)	Singleton vía <code>obtener()</code> con <code>get_or_create(pk=1)</code> , para que dos peticiones concurrentes no creen dos filas (:375-382). <code>_fecha()</code> recorta al último día del mes y evita que un 31 configurado reviente en meses cortos (:384-388)

App solicitudes (solicitudes/models.py)

Modelo	Campos clave	Constraints e índices	Notas de negocio
Notificacion	destinatario, tipo, titulo, mensaje, leida, fecha_lectura, solicitud (null)	notif_dest_leida_idx, notif_tipo_idx, notif_fecha_creacion_idx (:34-38)	La campana dentro de la aplicación; independiente del correo
EmailOutbox	asunto, cuerpo_texto, cuerpo_html, remitente, reply_to, destinatarios (JSON), estado, intentos, ultimo_error,	outbox_estado_disp_idx (:104-107)	Patrón outbox. MAX_INTENTOS = 5 (:77). La fila se escribe dentro de la misma transacción que el

	<code>clave_idempotencia</code> (unique, nullable), <code>disponible_en</code> , <code>enviado_en</code>		cambio de negocio; la garantía es <i>como máximo una vez por clave más al menos una vez por reintento</i> (:44-64). <code>disponible_en</code> implementa el backoff (:96-97)
TipoSolicitudCambio	<code>nombre</code> (unique), <code>codigo_estrategia</code> (null), <code>activo</code> , <code>genera_deuda</code>	<code>tipo_sol_activo_idx</code> (:129-131)	Tabla maestra de los seis tipos. <code>codigo_estrategia</code> es el primer nivel de búsqueda de la factory
SolicitudCambio	<code>explorador_solicitante</code> , <code>explorador_receptor</code> , <code>tipo_cambio</code> (FK), <code>estado</code> , <code>fecha_solicitud</code> , <code>fecha_cambio_turno</code> , <code>fecha_resolucion</code> , <code>fecha_cancelacion</code> , <code>aprobado_receptor</code> + <code>fecha</code> , <code>aprobado_supervisor</code> + <code>fecha</code> , <code>turno_origen/turno_destino</code> (SET_NULL), <code>solicitud_origen</code> , <code>reemplazada_por</code> , <code>snapshot_turnos_previos</code> , <code>snapshot_turnos_resultantes</code>	5 índices: <code>receptor+fecha+estado</code> , <code>turno_origen+estado</code> , <code>turno_destino+estado</code> , - <code>fecha_resolucion+estado</code> , <code>solicitante+-fecha_solicitud</code> (:207-233)	<code>'fecha_resolucion'</code> NO se sobrescribe al cancelar : de ella dependen la ventana de 30 minutos y el orden de la guardia LIFO; para eso existe <code>fecha_cancelacion</code> (:151-165). Los dos snapshots permiten revertir y detectar que alguien más tocó el día (:186-201)
CambioPermanenteDetalle	<code>solicitud</code> (OneToOne), <code>fecha_inicio</code> , <code>fecha_fin</code>	<code>camb_perm_solicitud_idx</code> , <code>camb_perm_fecha_inicio_idx</code> (:250-253)	<code>clean()</code> exige <code>fecha_fin >= fecha_inicio</code> (:255-258)
CambioPermanenteDia	<code>cambio_permanente</code> , <code>fecha_especifica</code> o <code>dia_semana</code> , <code>tipo</code>	3 índices + <code>CheckConstraint camb_perman_dia_tipo_valido</code> , que exige	<code>save()</code> rechaza sábado y domingo, tanto en <code>dia_semana</code> como

		exactamente uno de los dos campos según tipo (:314-328)	en fecha_especifica (:330-349). Sin filas se usa el rango completo, por retrocompatibilidad (:273)
DobladaDetalle	solicitud (OneToOne), minutos_deuda (30), fecha_pago (obligatoria), tipo_cesion , jornada_cedida , es_intercambio , jornada_pago_sabado (AM/PM/AMBAS), fecha_pago_semana , jornada_cubre_en_pago , submodalidad_semana , snapshot previo y resultante, empleado_receptor	doblada_solicitud_idx , doblada_fecha_pago_idx , doblada_receptor_fecha_idx (:470-474)	No existen dobladas abiertas: fecha_pago es obligatoria (:387-389). es_intercambio = True es un swap de días y no genera ni altera deudas (:403-408). submodalidad_semana distingue los sub-flujos de CAMBIO DESCANSO en temporada; las filas antiguas sin valor se tratan como intercambio_dia (:435-442). El historial excluye jornada_pago_sabado (:464)
DobladaPermanent eDetalle	solicitud (OneToOne), fecha_inicio , fecha_fin , dias_cesion/dias_devolucion (CSV 0-6), fechas_cesion/fechas_devolucion (CSV ISO), minutos_deuda , empleado_receptor , snapshots	dob_perm_solicitud_idx , dob_perm_fecha_idx (:537-540)	Las fechas específicas tienen prioridad sobre los días de la semana; si están vacías se expanden los weekdays (:504-514). No se permiten domingos (:490). clean() valida el rango (:554-557)
DeudaExplorador	deudor , acreedor , solicitud_origen ,	deuda_deudor_estado_idx ,	Deuda entre personas

	fecha_pago_pactada, fecha_pago_real, estado, media_jornada, jornada_cedida	deuda_acreedor_esta do_idx, deuda_fecha_pago_es tado_idx (:627-631)	
DeudaCorporativa	explorador, solicitud_origen (SET_NULL), minutos (30), fecha_doblada, estado (activa/pagada/cancelada), fecha_pago, comentario	deuda_corp_exp_esta do_idx, deuda_corp_fecha_es tado_idx, deuda_corp_fecha_do b_idx (:695-699)	Deuda con la empresa . obtener_deuda_total() suma las activa (:705-716)
ReprogramacionDiaDoblada	doblada_origen, explorador, fecha_original, jornada_debida, fecha_reprogramada, jornada_pago_previa, estado, motivo, registrado_por	reprog_explorador_e stado_idx, reprog_doblada_idx (:773-776)	Es simétrico (sirve para solicitante o receptor) y no afecta al otro explorador . El día original se anula por soft-delete y se le resta su deuda; al programar el nuevo se le vuelve a agregar la doblada y los 30 minutos (:719-729). jornada_pago_previa se restaura al cancelar (:754-757)
CierreSolicitudesConfig	habilitado, dia_cierre, hora_cierre	— (singleton)	Si habilitado=False no hay ninguna restricción (:802). obtener() usa get_or_create(pk=1) (:815-822)
CierreSemanaOverride	semana_lunes (unique), habilitado, dia_cierre, hora_cierre	unique en semana_lunes (:830)	Ajusta el cierre de una semana concreta sobre el default global

App permisos (permisos/models.py)

Modelo	Campos clave	Constraints e índices	Notas de negocio
PDH	explorador, solicitud (opcional), fecha, horas (decimal 5,2), supervisor, tipo_registro (pago_horas), comentario, deudas_pagadas (M2M a DeudaCorporativa), permisos_pagados (M2M a PermisoEspecial)	pdh_exp_fecha_idx, pdh_solicitud_idx, pdh_fecha_idx (:31-35)	clean() exige 0 < horas <= 24 (:37-42). Las horas del PDH son la suma de las deudas que salda (:18-21)
PermisoEspecial	empleado, tipo (5 valores), es_permanente, fecha_inicio, fecha_fin, dias_semana (CSV), tiempo, especificacion, cubre (SET_NULL), motivo, estado, pagado, fecha_pago, supervisor, comentario_supervisor, jornada_trabaja, fecha_compensacion, snapshot_turnos_previos	perm_esp_emp_estado_idx, perm_esp_estado_idx, perm_esp_fecha_idx (:141-145)	El tipo MEDIA_JORNADA_TEMPO RADA parte el día completo de temporada en dos y no genera deuda (tiempo=0, :90-99). horas_totales() multiplica por las ocurrencias si es permanente (:118-135). clean() valida el rango (:147-152)

5.3 Migraciones

solicitudes tiene 35 archivos en solicitudes/migrations/. El orden de dependencias entre apps es obligatorio y no se puede invertir: permisos/models.py:3 importa SolicitudCambio y solicitudes/models.py:4 importa Turno, así que el orden es empleados → turnos → solicitudes → permisos.

Migraciones delicadas — las que tocan invariantes, no solo columnas:

- La que añade Turno.activo_key y la UniqueConstraint turno_unico_activo_por_jornada

(`turnos/models.py:80-83`). Falla si la base ya tiene turnos activos duplicados; hay que limpiarlos antes.

- La que añade la `CheckConstraint` `turno_tipo_cambio_valido` (`turnos/models.py:90-94`).

Falla si existe algún `Turno.tipo_cambio` fuera de `TipoCambioTurno.TODOS`. El comando `actualizar_codigos_estrategia` normaliza el catálogo antes.

- La `UniqueConstraint`(`fecha`, `tipo`) de `DiaEspecial` (`turnos/models.py:148`): sin ella, el borrado por `año_planificacion` dejaba restos que se duplicaban al regenerar el año.

En producción no hay backfills previstos: la base arranca limpia (§ 1.3).

5.4 Consultas críticas y su coste

Consulta	Dónde	Protección aplicada
Guardia LIFO al cancelar: todas las solicitudes aprobadas posteriores de las dos personas	<code>solicitudes/use_cases/cancelar_solicitud.py:254-287</code>	<code>select_related('doblada', 'doblada_permanente', 'cambio_permanente') (:281)</code> evita un N+1 al calcular los pares afectados de cada candidata
Candidatos de cobertura: jornada base de todos los empleados activos	<code>solicitudes/views/api_fin_semana.py:226-230</code>	Se cargan en una consulta a <code>AsignarJornadaExplorador</code> y se indexan en el dict <code>bases</code> ; sin eso serían N consultas
"Mis Turnos" por mes	<code>turnos/api/views/turnos_mes.py</code>	Cacheado como <code>turnos_mes_<emp>_<año>_<mes></code> durante una hora e invalidado con <code>CacheService.invalidar_cache_turnos_empleado</code> (<code>config/settings.py:288-296</code>)

Barrido del outbox	<code>solicitudes/services/email_outbox_service.py</code>	Índice <code>outbox_estado_disp_idx</code> sobre (<code>estado</code> , <code>disponible_en</code>) (<code>solicitudes/models.py:106</code>)
Matriz empleado × día de CT permanente	<code>solicitudes/services/ct_permanente_helper.py</code>	Precarga en lote, introducida en el commit <code>00558d8</code>

⚠ La caché es **compartida obligatoriamente** en producción: con varios workers de Unicorn, `LocMemCache` invalidaría solo el proceso que atendió la petición y el resto seguiría sirviendo el mes viejo hasta una hora (`config/settings.py:279-296`).

6. Endpoints y URLs

Enrutado raíz en `config/urls.py`: `health/`, `health/ready/`, `admin/`, `'` (login), `dashboard/`, `empleados/`, `turnos/`, `permisos/` y `solicitudes/` con namespace (`config/urls.py:24-38`). `__debug__` solo existe con `DEBUG=True` (`config/urls.py:40-44`).

Todas las respuestas JSON se construyen con `json_ok` / `json_error` de `core/utils/json_responses.py`; `json_error` acepta `status` y un `code` textual que el frontend usa para distinguir el motivo.

6.1 Tabla de rutas

Raíz y core

Método	Ruta	Vista	Permiso	Respuesta
GET	<code>/health/</code>	<code>core.health.health</code> (<code>config/urls.py:27</code>)	ninguno	Estado, sin tocar la base
GET	<code>/health/ready/</code>	<code>core.health.readiness</code> (<code>config/urls.py:28</code>)	ninguno	Estado verificando la conexión
GET/POST	<code>/</code>	<code>LoginFormView</code> (<code>core/login/urls.py:5</code>)	ninguno	Formulario / sesión
POST	<code>/logout/</code>	<code>LogoutUserView</code> (<code>core/login/urls.py:6</code>)	sesión	Redirección
GET	<code>/dashboard/</code>	<code>DashboardView</code> (<code>core/dashboard/urls.py:5</code>)	sesión	Portada

—	/admin/	Django admin (config/urls.py:30)	is_staff	—
---	---------	-------------------------------------	----------	---

No existe ruta de recuperación de contraseña. `core/login/urls.py` declara exactamente dos rutas, `login` y `logout` (`core/login/urls.py:4-7`), y ninguna vista de reseteo de `django.contrib.auth` está incluida en `config/urls.py`. El desbloqueo lo hace un administrador. Ver § 16.

solicitudes/ (solicitudes/urls.py)

Grupo	Rutas	Permiso
Panel	<code>'', notificaciones-solicitudes/</code> (legacy, misma vista)	sesión
Notificaciones	<code>notificaciones/, notificaciones/<id>/marcar-leida/</code>	sesión
Mis cosas	<code>mis-solicitudes/, mis-favores/, solicitudes-pendientes/</code>	sesión
Gestión	<code>gestion-solicitudes/</code> y sus acciones <code>reenviar/, cancelar/, eliminar/</code>	supervisor
Reprogramación	<code>reprogramaciones/, .../registrar/<id>/, .../<id>/programar/, .../<id>/cancelar/</code>	supervisor
Cierre semanal	<code>cierre-solicitudes/</code>	supervisor
Aprobación web	<code>aprobar-solicitud/<id>/, rechazar-solicitud/<id>/, aprobar-solicitud-receptor/<id>/, rechazar-solicitud-receptor/<id>/, aprobar-solicitud-ambos/<id>/, cancelar-solicitud/<id>/</code>	sesión + rol comprobado en el caso de uso
Aprobación por correo	<code>aprobar-email/<id>/<token>/, rechazar-email/<id>/<token>/, aprobar-receptor-</code>	token HMAC, sin sesión

	email/<id>/<token>/, rechazar-receptor- email/<id>/<token>/	
Creación	procesar-solicitud/ (POST único de los seis formularios), cambio-turno/, cambio- turno/solicitar/<tipo_id>/	sesión
Consulta del formulario	obtener-empleados- disponibles/, obtener-turno- explorador/, obtener- jornadas-rango/, obtener- cambio-aprobado/, descansos- semana-usuario/, cambio- descanso-fines/, alternancia-mes/, dfds- companeros/, dobladas- semana/, cobertura- candidatos/, obtener- detalle-solicitud/<id>/	sesión
Previsualización	previsualizar-ct- permanente/, previsualizar- doblada-permanente/, dias- disponibles-doblada- permanente/	sesión
Doblada	obtener-exploradores- doblada/, verificar-doblada- existente/, exploradores- con-doblada/, sabado-pago- comprometido/, obtener- fechas-descanso/, verificar- coincidencia-jornadas/	sesión
Administración	tipos-solicitud/ + create/, edit/<pk>/, delete/<pk>/	supervisor

Las rutas de `permisos-detalle/` están comentadas en el archivo y **no existen** en runtime (`solicitudes/urls.py`, bloque "ADMINISTRACIÓN").

turnos/, empleados/, permisos/

Incluidas desde `config/urls.py:34-36`. Concentran los CRUD de calendario (días especiales, descansos de semana, asignación especial anual, apertura de año), de personas (salas, competencias, restricciones, sanciones, PDH) y de permisos especiales. La referencia de la fuente de verdad del calendario está en [05-referencia/turnos/](#) (./05-referencia/turnos/).

6.2 Fichas de endpoints no triviales

POST /solicitudes/procesar-solicitud/

- 22. **Método y ruta.** `POST /solicitudes/procesar-solicitud/` (`solicitudes/urls.py`, nombre `procesar_solicitud`).
- 23. **Vista y archivo.** `ProcesarSolicitudView` (`solicitudes/views/procesar_solicitud.py:13`).
- 24. **Para qué existe.** Es la **única** puerta de creación para los seis tipos de solicitud. El tipo se elige por `tipo_solicitud_id`, no por la ruta.
- 25. **Permiso.** `LoginRequiredMixin` (`solicitudes/views/procesar_solicitud.py:13`). No exige rol: cualquier explorador autenticado crea solicitudes.
- 26. **Entrada.** `tipo_solicitud_id` (int, obligatorio) más el `POST` completo del formulario, que varía por tipo; lo parsea `solicitudes/services/solicitud_request_parser.py`.
- 27. **Validaciones, en orden.** (a) `tipo_solicitud_id` presente, si no 400 (:18-19); (b) el tipo existe, si no 400 (:21-24); y dentro de `SolicitudOrchestrator.procesar` (`solicitudes/services/solicitud_orchestrator.py:579`): dedupe del `POST` idéntico (:59-81), cierre semanal sobre las fechas objetivo (:146-162), sanción del solicitante (:165-172), sanción del compañero (:174-187), restricción médica —advierte, no bloquea— (:293) y por último `SolicitudFactory.validar_solicitud` y `crear_solicitud`.
- 28. **Respuesta 201.** `json_ok({'message': ..., 'solicitud_id': ...}, status=201)`. La cobertura con dos compañeros devuelve `{'message': ..., 'solicitud_ids': [id, id]}` (`solicitud_orchestrator.py:286-291`).
- 29. **Errores.** 400 `missing_fields` (falta `tipo_solicitud_id`), 400 `invalid_type` (tipo inexistente), 400 `cierre_semanal`, 400 `creation_failed`, 403 `sancionado`, 403 `sancionado_receptor`, 409 `duplicate_request` (reenvío idéntico en 10 s), 500 `internal_error`.
- 30. **Efectos secundarios.** Crea `SolicitudCambio` en estado `pendiente` más su fila de detalle, y encola `EmailOutbox` en la misma transacción. **No toca `Turno`**: el calendario solo cambia al aprobar.
- 31. **Servicios que invoca.** `CrearSolicitudUseCase` → `SolicitudOrchestrator` → `SolicitudFactory` → la `Strategy` del tipo → los `validators`.
- 32. **Tests.** `solicitudes/tests/` (43 archivos), entre ellos `test_politica_temporada.py`, `test_cobertura_dos_companeros.py` y `test_cambio_doblada_candidatos.py`.
- 33. **Gotchas.** El dedupe usa una huella SHA-256 del `POST` completo menos el CSRF, con TTL de 10 s: dos solicitudes distintas del mismo tipo no chocan, solo el reenvío idéntico (`solicitud_orchestrator.py:59-81`). Y el **cierre semanal falla abierto**: si la comprobación revienta, la solicitud pasa y solo queda un `CRITICAL` en el log (:159-162).

POST /solicitudes/cancelar-solicitud/<solicitud_id>/

- 34. **Método y ruta.** `POST /solicitudes/cancelar-solicitud/<int:solicitud_id>/`.
- 35. **Vista y archivo.** `CancelarSolicitudView` (`solicitudes/views/aprobacion_views.py:57`).

- 36. **Para qué existe.** Deshacer una solicitud propia. Si estaba aprobada, revierte además el efecto ya aplicado al calendario.
- 37. **Permiso.** `LoginRequiredMixin` y el usuario debe tener `empleado (:60-61)`; el caso de uso comprueba que sea el solicitante.
- 38. **Entrada.** `solicitud_id` en la URL. Sin cuerpo.
- 39. **Validaciones, en orden.** (a) la solicitud es propia; (b) si está aprobada, han pasado 30 minutos o menos desde `fecha_resolucion` (`solicitudes/use_cases/cancelar_solicitud.py:20,216-219`); (c) **guardia LIFO:** no hay otra solicitud aprobada más reciente sobre alguno de los mismos (persona, día) (:254-287); (d) **guardia de integridad:** los turnos actuales coinciden con `snapshot_turnos_resultantes`, es decir, nadie más tocó esos días (:289-300).
- 40. **Respuesta 200.** `json_ok({'message': msg}) (:87)`.
- 41. **Errores.** `403 forbidden` (sin empleado o no es propia), `400 cambio_mas_reciente` (LIFO), `400 conflicto_integridad`, `400 ventana_expirada`, `400 invalid_state`, `500 internal_error (:61-72,90)`. El mapeo se hace **inspeccionando el texto del mensaje**, no un código.
- 42. **Efectos secundarios.** Restaura los turnos del snapshot, cancela las deudas generadas, transiciona a `cancelada`, escribe `fecha_cancelacion` sin tocar `fecha_resolucion`, borra las claves de contador en caché y crea la notificación de cancelación (:73-86).
- 43. **Servicios.** `CancelarSolicitudUseCase` → `_revertir_por_tipo (:474)` → el servicio de aplicación del tipo; después `CacheService.delete_many` y `NotificacionService.crear_notificacion_cancelacion`.
- 44. **Tests.** El commit `40a7ed8` cubre la reversión de las cinco opciones de temporada; el resto está en `solicitudes/tests/`.
- 45. **Gotchas.** El mapeo de error por subcadena (`'más reciente' in msg`, `'minutos' in msg`) acopla la vista al texto de los mensajes: cambiar la redacción de un mensaje del caso de uso cambia el código HTTP que ve el frontend.

POST /solicitudes/aprobar-solicitud-ambos/<solicitud_id>/

- 46. **Método y ruta.** `POST /solicitudes/aprobar-solicitud-ambos/<int:solicitud_id>/`.
- 47. **Vista y archivo.** `AprobarSolicitudAmbosView` (`solicitudes/views/aprobacion_views.py:93`).
- 48. **Para qué existe.** Cuando la misma persona es a la vez compañero receptor y supervisor, aprueba ambos roles en un solo paso.
- 49. **Permiso.** `LoginRequiredMixin` y `empleado` asociado (:96-97); el caso de uso valida que tenga efectivamente los dos roles sobre esa solicitud.
- 50. **Entrada.** `solicitud_id` en la URL; `comentario_respuesta` (str, opcional) en el POST (:99).
- 51. **Validaciones.** Delegadas a `AprobarAmbosRolesUseCase` (`solicitudes/use_cases/aprobar_solicitud.py:39`). Al completarse ambas aprobaciones, `SolicitudAprobacionService` llama `SolicitudFactory.revalidar_para_aprobar` antes de aplicar nada (`solicitudes/services/solicitud_aprobacion_service.py:120`).
- 52. **Respuesta 200.** `json_ok({'message': msg}) (:104)`.

- 53. **Errores.** `403 approval_error` si el mensaje contiene '`permisos`', `400 approval_error` en el resto, `500 internal_error (:101-107)`.
- 54. **Efectos secundarios.** Marca ambas aprobaciones con su fecha, transiciona a `aprobada`, fija `fecha_resolucion`, captura el snapshot previo, materializa los turnos, genera deudas y encola los correos.
- 55. **Servicios.** `AprobarAmbosRolesUseCase` → `SolicitudAprobacionService` → `SolicitudFactory.revalidar_para_aprobar` (`solicitudes/services/solicitud_factory.py:293,308`) → servicio de aplicación del tipo.
- 56. **Tests.** Suite de aprobación en `solicitudes/tests/`.
- 57. **Gotchas.** La revalidación puede fallar aunque la solicitud fuera legal al crearse: entre la creación y la aprobación el mundo cambió. Ese rechazo tardío es intencional, no un fallo.

GET /solicitudes/cobertura-candidatos/

- 58. **Método y ruta.** `GET /solicitudes/cobertura-candidatos/`.
- 59. **Vista y archivo.** `CoberturaCandidatosView` (`solicitudes/views/api_fin_semana.py:182`).
- 60. **Para qué existe.** Poblar el desplegable de compañeros de la opción "Que me cubran mi día" de CAMBIO DESCANSO en temporada, con el motivo por el que cada uno puede o no.
- 61. **Permiso.** `LoginRequiredMixin (:182)`. Un usuario sin `empleado` recibe lista vacía (`:202-204`).
- 62. **Entrada.** `fecha_trabajo` (ISO, obligatoria), `opcion` (AM|PM, obligatoria), `fecha_pago` (ISO, opcional) (`:206-218`).
- 63. **Validaciones, en orden.** (a) parámetros presentes y `opcion` en AM/PM, si no `400 (:216-218)`; y por cada candidato del grupo contrario (`:243`): (b) ya trabaja la jornada `opcion` ese día → descartado (`:249-251`); (c) ya tiene el día completo AM+PM → descartado (`:252-254`); (d) el día está comprometido en otra solicitud → descartado (`:255-257`); (e) no trabaja `opcion` el día de pago → descartado, porque no habría jornada que devolverle (`:258-266`).
- 64. **Respuesta 200.** `json_ok({'candidatos': [...]}),` con disponibilidad, motivo y la jornada del candidato el día que cubre y el día de pago (`:192-193,204`).
- 65. **Errores.** `400 bad_request`, con el texto `Parámetros inválidos (fecha_trabajo, opcion=AM|PM) (:217-218)`.
- 66. **Efectos secundarios.** Ninguno: es solo lectura.
- 67. **Servicios.** `TurnoService.dia_comprometido_por_solicitud` y `CambioDescansoAplicacionService.__jornadas_actuales (:245-248)`.
- 68. **Tests.** `solicitudes/tests/test_cambio_doblada_candidatos.py,`
`solicitudes/tests/test_cobertura_dos_companeros.py.`
- 69. **Gotchas.** El filtro (e) existe porque el desplegable y la validación de envío miraban fechas distintas: el selector solo el día de cesión y el backend además el día de pago, así que ofrecía compañeros que el envío tumbaba (`:258-263, commit 9c429e3`). Es el patrón 37 de `PROTECTION_PATTERNS.md`. Si tocas este endpoint, toca `_validar_semana_cobertura` en el mismo commit.

GET /solicitudes/aprobar-email/<solicitud_id>/<token>/ y sus tres hermanas

- 70. **Método y ruta.** GET, cuatro rutas: `aprobar-email/` y `rechazar-email/` (supervisor), `aprobar-receptor-email/` y `rechazar-receptor-email/` (compañero), todas con `<int:solicitud_id>/<str:token>/`.
- 71. **Vista y archivo.** `AprobarSolicitudEmailView` (`solicitudes/views/aprobacion_email.py:83`), `RechazarSolicitudEmailView` (`:142`), `AprobarSolicitudReceptorEmailView` (`:201`) y `RechazarSolicitudReceptorEmailView` (`:253`). Rutas en `solicitudes/urls.py:98` y siguientes.
- 72. **Para qué existe.** Aprobar o rechazar desde el correo con un clic, sin iniciar sesión.
- 73. **Permiso. Ninguno de sesión.** La autorización es el `token` de la URL, verificado por `tokens_aprobacion.verificar` (`solicitudes/views/aprobacion_email.py:140,199,251,303`; el mismo módulo respalda `EmailService._verificar_token`, `solicitudes/services/email_service.py:528`).
- 74. **Entrada.** `solicitud_id` (int) y `token` (str), ambos obligatorios y en la URL.
- 75. **Validaciones, en orden.** (a) firma y caducidad: `signing.loads(token, salt='solicitudes.aprobacion-email', max_age=...)`, que devuelve `False` ante firma inválida o token caducado (`solicitudes/services/tokens_aprobacion.py:84-91`); (b) el token debe ser de **esta** solicitud, de **este** rol y de la persona que **hoy** ocupa ese rol (`:96-107`); (c) idempotencia: `_ya_resuelto_para()` corta si la solicitud ya está resuelta o si ese rol ya respondió (`solicitudes/views/aprobacion_email.py:58-73`, llamado en `:115,174,226,278`).
- 76. **Respuesta 200.** Página HTML `solicitudes/aprobacion_exitosa.html` con el detalle rico de la solicitud, renderizada por `_render_resultado` (`solicitudes/views/aprobacion_email.py:49-55`). No JSON.
- 77. **Errores.** Token inválido o caducado → página de error. Solicitud inexistente → `get_object_or_404`. Enlace ya usado → la misma página con `ya_procesada=True` y sin reenviar notificaciones (`:54`).
- 78. **Efectos secundarios.** Los mismos que la aprobación web. En el caso ya resuelto, ninguno.
- 79. **Servicios.** `solicitudes/services/tokens_aprobacion.py` y los casos de uso de aprobación.
- 80. **Tests.** `solicitudes/tests/test_tokens_aprobacion.py` (25 tests: firma, caducidad, sal, rol y suplantación) más la suite de aprobación en `solicitudes/tests/`.
- 81. **Gotchas.** El token va firmado con `SECRET_KEY`: **todas las instancias deben compartir la misma** o el enlace fallará según a cuál encamine el ALB, y **rotar `SECRET\_KEY` invalida los enlaces ya enviados** (§ 9.2 y § 13.3). El uso único no se guarda en ninguna lista: depende del estado de la solicitud en base de datos, así que cualquier ruta que apruebe sin actualizar `estado/aprobado_*` reabre el enlace.

7. Servicios y lógica de dominio

7.1 Mapa de servicios

`solicitudes/services/` tiene 29 módulos. Los que importan, y quién los llama:

Servicio	Qué hace	Quién lo llama
<code>solicitud_orchestrator.py</code>	Encadena los chequeos transversales —dedupe, cierre, sanciones, restricción— y despacha al tipo. Contiene además los dos flujos multi-solicitud: <code>_procesar_cobertura_dos (:191)</code> y <code>_procesar_doblada_permanente_multi (:366)</code>	<code>CrearSolicitudUseCase</code>
<code>solicitud_factory.py</code>	Resuelve el <code>TipoSolicitudCambio</code> a su <code>Strategy</code> en cascada de cuatro niveles (<code>:123-187</code>) y expone <code>validar_solicitud</code> , <code>crear_solicitud</code> y <code>revalidar_para_aprobar (:293)</code>	orquestador y servicio de aprobación
<code>solicitud_aprobacion_service.py</code>	Ejecuta la aprobación: revalida (<code>:120</code>) y aplica	casos de uso de aprobación
<code>solicitud_request_parser.py</code>	Convierte el <code>POST</code> en el dict de datos que entienden las estrategias	orquestador
<code>cambio_descanso_aplicacion_service.py</code>	Materializa y revierte CAMBIO DESCANSO. Contiene <code>_marcar_reemplazadas (:310-349)</code> , donde vive "la última aprobada gana por día"	<code>CambioDescansoStrategy</code> , cancelación

<code>doblada_aplicacion_service.py</code> , <code>doblada_permanente_aplicacion_service.py</code> , <code>d_fds_aplicacion_service.py</code>	Lo mismo para DOBLADA, DOBLADA PERMANENTE y D FDS	sus estrategias
<code>doblada_snapshot_service.py</code>	Captura y restaura snapshots de turnos; reconciliación de días colaterales (:299,563)	servicios de aplicación
<code>deuda_service.py</code> , <code>deuda_corporativa_service.py</code> , <code>doblada_deuda_service.py</code> , <code>doblada_pago_service.py</code>	Generan, cancelan y saldan deudas	servicios de aplicación, PDH
<code>cierre_solicitudes_service.py</code>	Calcula la ventana de cierre semanal: <code>cutoff_para_fecha (:123)</code> , <code>fecha_bloqueada (:140)</code> , <code>validar_fechas (:148)</code>	<code>SolicitudOrchestrator.verificar_cierre</code>
<code>email_outbox_service.py</code>	Encola y reclama filas del outbox por UPDATE condicional	servicios de aplicación, comando <code>procesar_email_outbox</code>
<code>email_service.py</code>	Plantillas, enlaces firmados y backend SMTP	outbox
<code>tokens_aprobacion.py</code>	Fuente única de los tokens firmados de los enlaces de aprobación/rechazo por correo. <code>generar/verificar</code> para solicitudes de cambio (sal <code>solicitudes.aprobacion-email</code> , tipos <code>supervisor</code> y <code>receptor</code> , :44-46) y	<code>email_service.py:148,528</code> , las cuatro vistas de <code>views/aprobacion_email.py:140,199,251,303</code> y <code>permisos/services.py:30,34</code>

	<code>generar_permiso/verificar_permiso</code> para permisos especiales (sal <code>permisos.aprobacion-email, :116</code>), de modo que un token no vale en el circuito del otro. Firma con <code>django.core.signing</code> sobre <code>SECRET_KEY</code>, con caducidad (<code>:49-52</code>)	
<code>core/utils/error_token.py</code>	<code>render_error_token()</code>: punto único para la página de token inválido/caducado. Arma el contexto con <code>APPROVAL_LINK_MAX_AGE_DAYS</code> y con el <code>request_id</code>. <code>render_error_token_inesperado()</code> cierra un <code>except Exception</code> sin filtrar nada: deja la traza en el log y devuelve 500 con el mensaje genérico	las 15 llamadas de <code>views/aprobacion_email.py</code> y <code>permisos/views.py</code>
<code>core/utils/json_responses.py</code>	Formato único de respuesta de las APIs: <code>json_ok (:11)</code> y <code>json_error (:35, con code y extra)</code>. <code>json_error_inesperado(request, excepcion, mensaje) (:68)</code> cierra un <code>except Exception</code> sin filtrar nada: <code>logger.exception</code> con la traza (<code>:99</code>) y 500 con el mensaje propio del endpoint más <code>extra:</code>	todas las vistas API; las 7 llamadas al helper en <code>turnos/api/views/{dias_especiales, calculo_automatizado, turnos_mes}.py</code> y <code>solicitudes/views/api_turno_jornada.py</code>. Ver § 9.5

	<code>{request_id}</code> (:101-102)	
<code>notificacion_service.py</code>	Crea filas de <code>Notificacion</code>	vistas y casos de uso
<code>reprogramacion_doblada_service.py</code>	Anula el día no cumplido y programa el nuevo	vistas de reprogramación
<code>empleado_disponibilidad_service.py,</code> <code>doblada_filtro_service.py,</code> <code>solicitud_consulta_service.py,</code> <code>solicitud_context_service.py</code>	Alimentan desplegables y pantallas	vistas API
<code>ct_permanente_helper.py</code>	Expansión y filtrado de días de CT PERMANENTE	<code>CtPermanenteStrategy</code>
<code>permiso_service.py</code>	Permisos especiales	app <code>permisos</code>
<code>descanso_solicitud_service.py,</code> <code>fechas_helper.py,</code> <code>solicitud_service.py,</code> <code>solicitud_validator.py</code>	Apoyo transversal	varios

turnos/services/ (13 módulos) responde a "¿qué pasa realmente este día?":

`turno_service.py` (`estado_dia` / `estado_mes`, la fuente de verdad),
`descanso_semana_service.py`, `asignacion_especial_service.py`,
`alternancia_fines_semana_service.py`, `temporada_service.py`, `dia_especial_service.py`,
`festivos_rotacion_service.py`, `consolidado_horas_service.py`, `reporte_dia_service.py`,
`apertura_anio_service.py`, `jornada_service.py`, `doblada_turno_service.py` y
`turno_context_service.py`.

`solicitudes/domain/` contiene lo que no depende del ORM: `estado_machine.py`,
`bloqueo_partes.py`, `fechas.py`, `jornada.py`, `solicitud.py`. `solicitudes/repositories/` aísla
las consultas: `solicitud_repository.py` y `turno_repository.py`.

7.2 Validadores

Validador	Qué regla protege
-----------	-------------------

<code>services/validators/base_validator.py</code>	Reglas comunes a casi todos los tipos: día de mantenimiento con la precedencia de temporada (:66), <code>es_dia_temporada</code> (:394-409) y descanso de semana manual (:419)
<code>services/validators/ct_validator.py</code>	Reglas del cambio de turno sencillo
<code>services/validators/ct_permanente_validator.py</code>	Filtra los días no aptos del rango —fines de semana, festivos, mantenimiento, temporada, descansos y días comprometidos— y rechaza si no queda ninguno (:186,219)
<code>services/validators/doblada_validator.py</code>	Días especiales de doblada: los festivos de lunes a viernes y la temporada sí están permitidos (:173-197); y el caso del deudor que descansa el día de pago (:393)
<code>services/solicitud_validator.py</code>	Validación de nivel de solicitud, por encima del tipo

7.3 Tareas programadas, outbox y comandos de gestión

No hay Celery ni broker: el único trabajo diferido son los correos, resuelto con la tabla `EmailOutbox` más un cron que ejecuta un comando de gestión.

Comandos de `solicitudes` (`solicitudes/management/commands/`):

Comando	Para qué	Frecuencia
<code>procesar_email_outbox</code>	Envía las filas pendientes del outbox y reintenta las fallidas	cron, continuo
<code>instalar_calendario_colombiano</code>	Carga los festivos	una vez / anual
<code>actualizar_codigos_estrategia</code>	Sincroniza <code>TipoSolicitudCambio.codigo_estrategia</code>	tras tocar el catálogo
<code>archivar_solicitudes_antiguas</code>	Mueve solicitudes viejas	periódico
<code>cancelar_deudas_fin_semana, cancelar_deudas_huerfanas</code>	Saneamiento de deudas	puntual
<code>reaplicar_doblada, corregir_doblada_cesion_total</code>	Reparación manual	puntual

verificar_doblada, verificar_integridad_dobladas, verificar_efecto_aplicado, validar_dobladas_junio, validar_jornadas, test_factory, test_verificar_doblada_jeison, validar_fix_doblada_jeison	Diagnóstico	manual
---	-------------	--------

Comandos de `turnos` (turnos/management/commands/): `archivar_turnos_antiguos,`
`limpiar_festivos_futuros,` `materializar_alternancia,` `verificar_apertura_anio.`

⚠ Ocho de los comandos de `solicitudes` son de diagnóstico puntual, cuatro con nombres de incidencias concretas (`..._jeison`, `..._junio`). No son parte del ciclo de vida del sistema; ver § 16.

El manual de operación del outbox está en [MANUAL_OUTBOX_CORREOS.md](#) (./05-referencia/deployment/MANUAL_OUTBOX_CORREOS.md).

8. Reglas de negocio

8.1 Estados de una solicitud

```
stateDiagram-v2
    [*] --> pendiente : crear
    pendiente --> aprobada : receptor + supervisor
    pendiente --> rechazada : cualquiera rechaza
    pendiente --> cancelada : el solicitante se arrepiente
    aprobada --> cancelada : ventana de 30 min + guardias
    aprobada --> reemplazada : otra aprobada pisa el mismo dia
    aprobada --> pagada : se salda la deuda (PDH)
    rechazada --> [*]
    cancelada --> [*]
    reemplazada --> [*]
    pagada --> [*]
```

Qué muestra. Los seis estados de `SolicitudCambio` y las únicas transiciones legales. El mapa está en un solo sitio (`solicitudes/domain/estado_machine.py:11-18`) y la función `transicionar` lo hace cumplir, lanzando `EstadoTransicionError` cuando el destino no está permitido (:28,47-53). Cuatro estados son terminales y no tienen salida: `rechazada`, `cancelada`, `reemplazada` y `pagada` (:14-17). Se lee así: desde `pendiente` se puede ir a `aprobada`, `rechazada` o `cancelada`; una vez `aprobada` solo caben tres finales —cancelarla dentro de la ventana, que otra la reemplace, o que se pague su deuda—. Nada vuelve atrás.

8.2 Principios transversales

Cinco reglas atraviesan los seis formularios. Son el núcleo del dominio.

#	Regla	Dónde se aplica	Test que la cubre	Qué pasa si se viola
P 1	La última aprobada gana por día. El estado efectivo de un día es lo último aprobado que lo modifica. No se encadenan cambios: la solicitud anterior sobre ese (persona, día) pasa a reemplazada , con reemplazada_por apuntando a la nueva	solicitudes/services/cambio_descanso_aplicacion_service.py:310-349 , invocado en :408 ; el estado REEMPLAZADA se define en core/constants.py (EstadoSolicitud)	Suite de reemplazos en solicitudes/tests/	El día queda con dos solicitudes vigentes que se contradicen y el consolidado cuenta doble
P 2	No se encadenan cambios de descanso. Solo se cede el descanso de temporada original; un intercambio nuevo cancela el anterior en vez de apilarse	cambio_descanso_aplicacion_service.py:373 (documenta por qué no se marcan reemplazos cuando la solicitud no estaba aplicada) y services/strategies/cambio_descanso_strategy.py:444	solicitudes/tests/test_politica_temporada.py	Se pierde el rastro de a quién pertenece el descanso original
P 3	Los días de descanso de temporada solo se tocan desde	Regla y motivo en turnos/services/descanso_semanal_service.py:42-60 (es_dia_descanso_temporada) ; rechazo en	solicitudes/tests/test_politica_temporada.py:222	Un formulario ajeno cede

	CAMBIO DESCANSO. Son dos fechas por semana, no la temporada entera: el resto de la temporada sigue disponible para los demás formularios	solicitudes/services/strategies/doblada_strategy.py:189-197		un descanso fijado y el cómputo semanal de CAMBIO DESCANSO deja de cuadrar
P 4	Reversión al cancelar. Cancelar restaura snapshot_turnos_previos y cancela las deudas generadas, dentro de una ventana de 30 minutos desde fecha_resolucion, y solo si pasan la guardia LIFO y la de integridad	solicitudes/use_cases/cancelar_solicitud.py:20 (VENTANA_CANCELACION_MINUTOS = 30), :206-249, :254-287 (LIFO), :289-300 (integridad), :474 (_revertir_por_tipo)	Commit 40a7ed8: reversión de las cinco opciones de temporada	Se pisa el cambio de otra persona: se restaura un turno que el compañero ya no tiene
P 5	Re-validación al aprobar. Antes de aplicar nada se vuelve a validar contra el estado actual, porque entre la	solicitudes/services/solicitud_aprobacion_service.py:120 → solicitudes/services/solicitud_factory.py:293,308 → services/strategies/base_strategy.py:84	Suite de aprobación en solicitudes/tests/	Se aplica un acuerdo que ya era ilegal

creación y la aprobación el mundo pudo cambiar			cuan do se apro bó
--	--	--	--------------------

8.3 Reglas por área

Regla	Dónde se aplica	Qué pasa si se viola
Un sancionado no participa en ninguna solicitud, ni como compañero: si no, bastaría con que otro la enviara en su nombre	<code>solicitud_orchestrator.py:165-172</code> (solicitante), <code>:174-187</code> (receptor)	La sanción es evitable
Una restricción médica advierte, no bloquea: el usuario confirma	<code>solicitud_orchestrator.py:293</code> (<code>verificar_restriccion(..., confirmar)</code>)	—
El mismo POST no se procesa dos veces en 10 segundos	<code>solicitud_orchestrator.py:59-81</code>	Solicitud duplicada por doble clic
No existen dobladas abiertas: <code>fecha_pago</code> es obligatoria	<code>solicitudes/models.py:387-389</code>	Deuda sin vencimiento
Cada doblada acumula 30 minutos de deuda corporativa	<code>solicitudes/models.py:641,663-666</code>	El consolidado no refleja lo trabajado
Un intercambio de dobladas (<code>es_intercambio=True</code>) no genera ni altera deudas: es un swap de días	<code>solicitudes/models.py:403-408</code>	Deuda inventada
CT PERMANENTE solo de lunes a viernes	<code>solicitudes/models.py:330-349</code> (<code>CambioPermanenteDia.save</code>)	Turnos permanentes en fin de

		semana
DOBLADA PERMANENTE no permite domingos	<code>solicitudes/models.py:490</code>	Ídem
Un solo turno activo por (explorador, fecha, jornada)	<code>turnos/models.py:80-83</code> , garantizado por la base de datos	La persona aparece dos veces en la misma jornada y el consolidado cuenta doble
Turno.tipo_cambio solo acepta los ocho valores de TipoCambioTurno	<code>turnos/models.py:90-94</code>	Un typo entra en silencio y el turno deja de contarse en los filtros de texto exacto
La temporada manda sobre el mantenimiento	<code>turnos/models.py:186-200</code> (<code>es_mantenimiento_efectivo</code>)	Un lunes de temporada se trata como descanso
Una fecha sin AsignacionEspecialM anual es "sin planificar" y así se reporta; nunca se infiere un grupo	<code>turnos/models.py:284-296</code>	El pasado se recalcula solo
Un turno anulado no cuenta como falta ni genera deuda	<code>turnos/models.py:52-53, 27-36</code>	Se penaliza un día que se anuló
Una sanción no se borra: se levanta, dejando quién, cuándo y por qué	<code>empleados/models.py:193-208, 286-299</code>	Se pierde el hecho disciplinari

		o
Los roles Supervisor y Explorador se comparan exacto y están protegidos de renombrado y borrado	<code>empleados/models.py:73-85,98-107</code>	Escalada de privilegios; ya ocurrió con la búsqueda <code>icontains</code>
Las jornadas AM y PM no se pueden eliminar	<code>empleados/models.py:19-23,38-41</code>	El motor de turnos deja de resolver la jornada contraria
El cierre semanal bloquea las solicitudes cuyo objetivo caiga en la ventana; con <code>habilitado=False</code> no hay restricción	<code>solicitudes/models.py:792-803, solicitudes/services/cierre_solicitudes_service.py:140,148</code>	Se reprograma el fin de semana después de publicarlo
Un PDH pide entre 0 y 24 horas	<code>permisos/models.py:37-42</code>	Pago imposible
El permiso <code>MEDIA_JORNADA_TEMPO RADA</code> no genera deuda (<code>tiempo=0</code>)	<code>permisos/models.py:90-99</code>	Deuda duplicada

El detalle formulario a formulario está en [05-referencia/solicitudes/](#) (./05-referencia/solicitudes/) y, en lenguaje de usuario, en [manual_usuario.md](#) (./manual_usuario.md).

9. Autenticación, autorización y seguridad

9.1 Modelo de usuario y roles

El usuario es el `auth.User` estándar de Django, extendido por composición: `Empleado` tiene un `OneToOneField` a `User` (`empleados/models.py:45`). No hay `AUTH_USER_MODEL` personalizado. Casi todas las vistas usan `request.user.Empleado`, y varias comprueban antes `hasattr(request.user, 'Empleado')`, porque un `User` sin `Empleado` —un superusuario creado

con `createsuperuser`, por ejemplo— rompería el acceso (`solicitudes/views/aprobacion_views.py:60-61,96-97`).

Los roles son filas de `Role` unidas por `EmpleadoRole`. Dos son estructurales: `Supervisor` y `Explorador` (`empleados/models.py:82-85`). Qué puede hacer cada rol está en [manual_usuario.md](#) (`./manual_usuario.md`); aquí solo el mecanismo.

9.2 Permisos: dónde se comprueban

Capa	Mecanismo	Archivo
Sesión	<code>LoginRequiredMixin</code> de Django en cada vista	<code>solicitudes/views/*.py</code>
Rol, vistas HTML	<code>AdminRequiredMixin</code> — lanza o redirige	<code>core/mixins.py:44,57</code>
Rol, endpoints JSON	<code>SupervisorApiRequiredMixin</code> — devuelve JSON en vez de redirigir	<code>core/mixins.py:73,80</code>
Función común	<code>es_supervisor(user)</code> — la usan los dos mixins y el resto del código	<code>core/mixins.py:12-17</code>
Rol sobre una solicitud concreta	Los casos de uso: solo el solicitante cancela, solo el receptor aprueba como receptor	<code>solicitudes/use_cases/</code>
Token de correo	Token firmado con <code>django.core.signing</code> sobre <code>SECRET_KEY</code> , con caducidad y sal por circuito	<code>solicitudes/services/tokens_aprobacion.py:55-107,119-148</code>

Tokens de aprobación por correo

Los enlaces de "aprobar" y "rechazar" del correo actúan **sin sesión iniciada**: el token de la URL es la única credencial. Toda la firma vive en un módulo único, `solicitudes/services/tokens_aprobacion.py`; las seis copias que antes duplicaban la verificación —una en `email_service.py`, cuatro en `views/aprobacion_email.py` y una en `permisos/services.py`— ahora delegan en él (§ 16.3, B1).

Aspecto	Cómo funciona	Evidencia
Firma	<code>signing.dumps</code> / <code>signing.loads</code> , HMAC-	<code>tokens_aprobacion.py:59-62,84</code>

	SHA256 sobre <code>SECRET_KEY</code> más marca de tiempo	
Caducidad	<code>max_age = APPROVAL_LINK_MAX_AGE_DAYS × 86400</code> , por defecto 30 días	<code>tokens_aprobacion.py:49-52</code> , <code>config/settings.py:262</code>
Aislamiento por circuito	Sal <code>solicitudes.aprobacion_email</code> para solicitudes de cambio, <code>permisos.aprobacion_email</code> para permisos especiales: un token no sirve en el otro circuito aunque comparta clave	<code>tokens_aprobacion.py:44,116</code>
Vínculo con la persona	El token solo vale para quien hoy ocupa el rol; si al explorador le cambian de supervisor, el enlace del anterior deja de servir	<code>tokens_aprobacion.py:65-71,96-107</code>
Fallo cerrado	Firma inválida, token caducado, de otra solicitud o de otro rol → <code>False</code> , nunca excepción	<code>tokens_aprobacion.py:74-107</code>
Uso único	Sin lista de tokens gastados: lo impide <code>_ya_resuelto_para()</code> , que lee el estado de la solicitud en base de datos —el almacén compartido entre instancias— y también cubre el cliente de correo que pre-carga el enlace	<code>solicitudes/views/aprobacion_email.py:58-73</code>
Cobertura	25 tests dedicados	<code>solicitudes/tests/test_tokens_aprobacion.py</code>

Condición de despliegue. La verificación es *stateless*: no consulta base de datos ni caché, así que cualquier instancia detrás del balanceador valida un token emitido por otra. El precio es que **todas las instancias deben compartir la MISMA `SECRET_KEY`** —una sola entrada en Secrets Manager o SSM, nunca un valor generado por tarea—; si no, los enlaces fallan de forma intermitente según a qué instancia encamine el ALB. Además, **rotar `SECRET_KEY` invalida los enlaces ya enviados**: es el comportamiento correcto ante una filtración, pero hay que contarlo al planificar rotaciones rutinarias (los pendientes se resuelven entrando a la aplicación). Ver § 13.3.

Esa condición ya se cumple en las dos rutas de despliegue documentadas (auditado 2026-08-09):

Evidencia	Dónde	Qué demuestra
<code>SECRET_KEY = env('SECRET_KEY'),</code> sin valor por defecto	<code>config/settings.py:32</code>	La aplicación no arranca sin la clave: <code>django-envi</code> lanza <code>ImproperlyConfigured</code> . Una tarea mal configurada muere, no se inventa una clave
Cero apariciones de <code>get_random_secret_key</code> , <code>secrets.token*</code> o <code>urandom</code> en todo el repositorio	—	El código no puede generar una clave por instancia
<code>aws secretsmanager create-secret --name swalp/SECRET_KEY</code> y bloque <code>secrets:</code> del task definition	checklist Fargate (./05-referencia/deployment/CHECKLIST_DESPLIEGUE_FARGATE.md) fases 3 y 7.2	Una sola entrada leída por todas las tareas, también al subir <code>--desired-count</code> a 2 o más
Instancia única con Elastic IP, sin ALB, con un <code>.env</code> en la máquina	checklist EC2+RDS (./05-referencia/deployment/CHECKLIST_DESPLIEGUE_AWS_RDS.md)	Sin reparto de tráfico no hay nada que pueda divergir
El <code>Dockerfile</code> usa <code>SECRET_KEY=build-only</code> como prefijo de un <code>`RUN`</code> , no como <code>ENV</code>	<code>AppTurnosExplora/Dockerfile</code>	La clave ficticia del build no queda en el runtime de la imagen

Queda como comprobación operativa —no del repositorio— confirmar en la consola de AWS que el secreto es efectivamente uno solo el día del despliegue. El checklist de § 13.3 lo recoge.

`AUTHENTICATION_BACKENDS` pone `axes.backends.AxesStandaloneBackend` **antes** del backend de Django, para que el bloqueo por intentos se evalúe primero (`config/settings.py:229-232`).

Bloqueo por intentos fallidos (`django-axes`, `config/settings.py:330-334`):

Ajuste	Valor	Efecto
<code>AXES_FAILURE_LIMIT</code>	5	Se bloquea al quinto intento

		fallido
<code>AXES_COOLOFF_TIME</code>	<code>1</code>	Una hora de espera
<code>AXES_LOCKOUT_PARAMETERS</code>	<code>['username']</code>	Bloquea por usuario, no por IP
<code>AXES_RESET_ON_SUCCESS</code>	<code>True</code>	Un acierto limpia el contador
<code>AXES_VERBOSE</code>	<code>False</code>	Menos ruido en el log

Axes se desactiva bajo tests, porque exige un `request` en `authenticate()` que `client.login()` no provee (`config/settings.py:336-341`).

No hay recuperación de contraseña: ver § 6.1 y § 16.

9.3 Sesiones, CSRF, CSP y cabeceras

Ajuste	Valor	Dónde
<code>SESSION_COOKIE_HTTPONLY</code>	<code>True</code>	<code>config/settings.py:438</code>
<code>SESSION_COOKIE_SAMESITE</code> / <code>CSRF_COOKIE_SAMESITE</code>	<code>Lax</code>	<code>config/settings.py:439-440</code>
<code>SECURE_CONTENT_TYPE_NOSNIFF</code>	<code>True</code> (siempre, dev y prod)	<code>config/settings.py:432</code>
<code>X_FRAME_OPTIONS</code>	<code>DENY</code>	<code>config/settings.py:433</code>
<code>CSRF_FAILURE_VIEW</code>	<code>core.errors.csrf_failure</code> — vista propia, no la de Django	<code>config/settings.py:445</code>
<code>SECURE_SSL_REDIRECT</code> + <code>SECURE_PROXY_SSL_HEADER</code>	Solo si <code>SECURE_HTTPS</code>	<code>config/settings.py:453-457</code>
<code>CSRF_TRUSTED_ORIGINS</code>	de <code>env.list</code> , vacío por defecto	<code>config/settings.py:38</code>
<code>CORS_ALLOWED_ORIGINS</code>	de <code>env.list</code> , <code>localhost</code> por defecto	<code>config/settings.py:331-334</code>
<code>CORS_ALLOW_CREDENTIALS</code>	<code>True</code>	<code>config/settings.py:335</code>
<code>AUTH_PASSWORD_VALIDATORS</code>	los 4 estándar de Django	<code>config/settings.py:200-204</code>

El `csrftoken` **no** es `HttpOnly` a propósito: `api-client.js` lo lee con `document.cookie` para mandarlo en la cabecera `X-CSRFToken` de las llamadas AJAX (`config/settings.py:435-438`).

CSP. Hay dos políticas. La **activa** (`CONTENT_SECURITY_POLICY`, `config/settings.py:376-401`) todavía permite `cdn.jsdelivr.net`, `cdnjs.cloudflare.com`, `fonts.googleapis.com`, `fonts.gstatic.com` y `code.ionicframework.com`. La **objetivo** (`CONTENT_SECURITY_POLICY_REPORT_ONLY`, `:409-427`) ya solo deja Google Fonts, y se publica

como `Content-Security-Policy-Report-Only`: informa de las violaciones en la consola del navegador sin bloquear nada. Cuando se confirme que no aparece ninguna, basta con mover ese diccionario a `CONTENT_SECURITY_POLICY` y borrar el permisivo (:366-375).

Ambas mantienen `'unsafe-inline'` en `script-src` y `style-src` por los inline existentes; quitarlo exige migrar esos bloques a archivos o usar nonces (:362-364). Las dos declaran `frame-ancestors: 'none'` y `connect-src: 'self'`.

Regla operativa: al añadir un recurso externo nuevo hay que añadirlo a las **dos** allowlists de `config/settings.py`, o el navegador lo bloqueará. Pero el sentido de la migración es el contrario: autohospedar en `static/plugins/` en lugar de ampliar la lista (`config/settings.py`:366-375).

Las páginas de error (§ 9.5) llevan su CSS y su SVG **en línea**, así que dependen de `'unsafe-inline'` en la política activa; no añaden ningún origen externo, por lo que también pasan la política estricta en report-only. Si algún día se eliminan los inline en favor de nonces, estas cinco plantillas hay que revisarlas a mano (`templates/errors/_base_error.html`:32-35).

9.4 Datos sensibles y qué nunca se registra

Lo que el sistema guarda de personas: nombre, apellido, cédula (unique), correo, `RestriccionEmpleado.recomendacion` —texto libre de contenido médico— y `SancionEmpleado.motivo`. Ninguno de los dos últimos debe aparecer en logs ni en respuestas JSON de propósito general.

Los logs del orquestador registran **identificadores, no contenido**: `logger.info("Solicitud %d creada — tipo=%s solicitante=%s receptor=%s", ...)` (`solicitudes/services/solicitud_orchestrator.py`:672) y `logger.warning('Cobertura con 2 compañeros revertida (solicitante=%s): %s', solicitante.id, e)` (:275-276). Mantén ese criterio: identificadores sí; motivos médicos, contraseñas y tokens, nunca.

Las vistas capturan las excepciones y devuelven un mensaje genérico al cliente, dejando el traceback solo en el log con `logger.exception` (`solicitudes/views/procesar_solicitud.py`:28-30).

9.5 Páginas de error y trazabilidad de la petición

El razonamiento completo, con las alternativas descartadas, está en [ADR 007](#) (./03-arquitectura/adr/007-paginas-de-error-propias-y-request-id.md). Aquí va lo que hay que saber para trabajar con el sistema.

El problema. Con `DEBUG=True`, Django respondía a cualquier URL inexistente con su pantalla técnica, que imprime el **URLconf completo** —todos los endpoints, incluidos `aprobar-solicitud` y `rechazar-solicitud`—, y a un 500 con traceback, código fuente y variables locales. Es divulgación de información: CWE-215, CWE-209 y OWASP A05:2021 Security Misconfiguration. El caso concreto que lo destapó está escrito en el docstring del test de regresión (`core/tests/test_paginas_error.py`:8-10).

Las piezas. Todo vive en `core/errors.py`:

Pieza	Dónde	Qué hace
<code>RequestIDMiddleware</code>	<code>core/errors.py:51-76</code>	Genera <code>uuid4().hex[:12].upper()</code> , lo guarda en un <code>ContextVar</code> y en <code>request.request_id</code> , y lo devuelve en la cabecera <code>X-Request-ID</code>
<code>get_request_id()</code>	<code>core/errors.py:46-48</code>	Identificador de la petición en curso, o '-' fuera de una petición
<code>RequestIDFilter</code>	<code>core/errors.py:79-84</code>	<code>logging.Filter</code> que inyecta <code>record.request_id</code> para que los formatters lo impriman
<code>_render</code>	<code>core/errors.py:96-111</code>	Renderiza la plantilla <code>sin request</code> , con fallback si la propia plantilla falla
<code>bad_request / permission_denied / page_not_found / server_error</code>	<code>core/errors.py:114-131</code>	<code>handler400/403/404/500</code> , enlazados en <code>config/urls.py:56-59</code>
<code>csrf_failure</code>	<code>core/errors.py:154-164</code>	<code>CSRF_FAILURE_VIEW</code> (<code>config/settings.py:445</code>)
<code>previsualizar_error</code>	<code>core/errors.py:134-151</code>	Renderiza una página a demanda para revisarla en desarrollo
Plantillas	<code>templates/errors/_base_error.html</code> + <code>templates/{400,403,403_csrf,404,500}.html</code>	Una base y cinco instancias

El identificador de petición. `RequestIDMiddleware` es el **primer** middleware (`config/settings.py:71`): lo que quede por encima no queda etiquetado. Tres decisiones que no se deben deshacer:

- **No se acepta un `X-Request-ID` entrante del cliente.** Sería un valor controlado por el

atacante escrito en los logs: inyección de log y envenenamiento de trazas (`core/errors.py:58-61`). Hay test (`core/tests/test_paginas_error.py:84-91`).

- **Se usa un `ContextVar`**, no solo un atributo del `request`, porque el filtro de logging no recibe el `request` (`core/errors.py:41-43`).
- **Es aleatorio, no correlativo**: no revela cuántas peticiones ha atendido el sistema ni permite adivinar el de otro usuario (`core/errors.py:28-29`).

El usuario ve ese código en la página de error como «código de referencia», y es exactamente el mismo que viaja en la cabecera y en cada línea de log de esa petición (`core/tests/test_paginas_error.py:68-76`).

Por qué `_render` no recibe el `request`. Usa `loader.get_template(...).render(context)` y no `render(request, ...)` (`core/errors.py:96-100`). `render()` activaría los context processors, y `core.context_processors.permisos` consulta la base de datos: en un `handler500` provocado precisamente por la BD caída, eso lanzaría una segunda excepción y Django devolvería su 500 de emergencia en texto plano (`core/errors.py:90-94`). Además Django renderiza `handler500` con contexto **vacío** —sin `request`, sin `user`, sin context processors—, así que las plantillas no pueden depender de ninguna variable obligatoria. Si aun así la plantilla falla, hay una última red que registra la excepción y devuelve un HTML mínimo (`core/errors.py:101-110`).

Por qué las plantillas son autocontenidas. No heredan de `base.html` y no cargan **nada** externo: CSS y SVG en línea (`templates/errors/_base_error.html:6-30`).

Razón	Qué evita
Seguridad	No se muestra la ruta pedida, ni la excepción, ni nombres de vista. Solo el código opaco (<code>core/errors.py:124-126</code>)
Robustez	Heredar de <code>base.html</code> renderizaría el sidebar, que consulta <code>user.Empleado.notificaciones_no_leidas_count</code> y tocaría la BD posiblemente caída
Disponibilidad	La página se ve aunque WhiteNoise/S3 o la red de estáticos estén caídos

El fallo de CSRF no dice por qué falló. `csrf_failure` registra el `reason` con `logger.warning` y al usuario le muestra el mensaje genérico de sesión expirada (`core/errors.py:154-164`): decirle a un atacante si falló por «CSRF token missing» o por «Referer checking failed» le indica qué comprobación esquivar en el siguiente intento.

Cómo se revisan las páginas en desarrollo. Con `DEBUG=True` Django nunca llega a usar los handlers. Por eso se enruta `/__error__/<codigo>/` solo bajo `DEBUG` (`config/urls.py:38-48`):

`/__error__/400/`, `/403/`, `/404/`, `/500/` y `/419/` —código libre para distinguir el CSRF del 403 genérico— (`core/errors.py:142-148`). Esa ruta **no existe en producción**.

El código de referencia en los envíos sin recarga. Las páginas de error solo aparecen cuando el navegador *navega*. Los seis formularios de solicitud envían por `fetch` y no recargan: ante un 500 pintaban su propio aviso y el identificador se perdía, aunque el servidor lo mandaba en la cabecera `X-Request-ID` de esa misma respuesta (`core/errors.py:51-76`). Justo el caso más grave —una solicitud que pudo quedar a medias en la base— era el que se quedaba sin código. Lo cierra `static/js/utils/codigo-referencia.js`.

Decisión	Dónde	Por qué
Se envuelve <code>`window.fetch`</code> en lugar de tocar cada envío	<code>static/js/utils/codigo-referencia.js:89-107</code>	Hay ~50 llamadas a <code>fetch</code> repartidas por los formularios, casi ninguna usa <code>ApiClient</code> y varias descartan la respuesta en su propio <code>.catch()</code> . Envolver una vez las captura todas sin tocar la lógica de envío
Solo se guarda el identificador de respuestas con error (<code>!response.ok</code>)	<code>:94-96</code>	Si se guardara el de cualquier respuesta, un sondeo de fondo correcto pisaría el código del fallo que el usuario acaba de ver
Vigencia de 60 s (<code>VIGENCIA_MS</code>)	<code>:36</code> , <code>:40-44</code>	Pasado ese plazo el código se considera de otro incidente y no se muestra: un código equivocado manda al equipo a leer la traza de otra petición, y eso es peor que no dar ninguno
Se carga antes que jQuery y el resto de scripts	<code>templates/base.html</code> (jQuery después)	Un script que se registre después no captaría las peticiones que ya hayan salido
Un stub define <code>window.CodigoReferencia</code> antes de cargar el fichero	<code>templates/base.html</code> , justo encima del <code><script src=...></code>	Sin él, si el fichero no llegara a cargarse los <code>.catch()</code> lanzarían <code>ReferenceError</code> antes de <code>rehabilitar()</code> / <code>restablecer()</code> : el usuario se quedaría sin aviso y con el formulario bloqueado. Una ayuda de diagnóstico no puede ser un punto de fallo en la propia ruta de error. El fichero real sobrescribe el stub al cargarse

Salvaguardas del shim, que **no se deben deshacer** (§ 16.1): no altera argumentos ni respuesta — devuelve la misma **Response** intacta (:101)—, **no encadena** `.catch``, así que un fallo de red sigue rechazando la promesa igual que antes (:103-105), y protege la lectura de la cabecera con `try/catch` para que una respuesta opaca no rompa la petición (:93-100).

La API tiene cuatro métodos: `ultimo()` devuelve el código vigente o `null` (:40-44); `mensaje(txt)` lo añade en **texto plano**, para los avisos que van en `text`: de `SweetAlert` (:53-58); `htmlMensaje(html)` lo añade como marcado, para los que se inyectan en `html`: — `notificar()` pasa su tercer argumento ahí, donde un `\n` no se vería— y sanea el valor con `/[^A-Za-z0-9]/g` antes de interpolarlo (:70-79); `registrar(codigo)` permite alimentarlo a mano si algún día se usa XHR (:82-86).

Ocho puntos de aviso de **envío fallido** en los seis `static/js/cambio-turno/solicitar_*.js` lo incorporan (p. ej. `solicitar_cambio_turno.js:421`, `solicitar_d_fds.js:442`, `solicitar_doblada_permanente.js:705`, `solicitar_cambio_descanso.js:1328,1362,1382`). Siete usan `htmlMensaje`, de modo que el código sale en el mismo recuadro monoespaciado en los seis formularios; `solicitar_ct_permanente.js:1748` y `solicitar_doblada.js:2862` pasaron de `text`: a `html`: justamente para no ser la excepción. El único que usa `mensaje()` es `solicitar_cambio_turno.js:421`, porque ahí el texto se compone antes de decidir en qué aviso se pinta y además se inspecciona con `includes()`. Ojo con ese punto: el código se aplica **sobre** `data.error || ...`, no solo sobre el mensaje de respaldo; envolver únicamente el respaldo hacía que el código se perdiera cada vez que el servidor mandaba texto propio. Los avisos de **carga** de listas (compañeros, dobladas) **no lo llevan a propósito**: no hay nada que reportar, se reintenta solo al cambiar de selección. El CSS del aviso está en `static/css/base_custom.css:23-36`: `white-space: pre-line` en `.swal2-html-container`, porque los mensajes que llegan por `text`: traen saltos de línea reales, y el estilo del `<code>` del código.

La otra vía sin plantilla: los enlaces de aprobación por correo.

`templates/solicitudes/error_token.html` es una respuesta normal de una vista, no un handler, así que quedó fuera del barrido inicial. Cuatro `except Exception` de `solicitudes/views/aprobacion_email.py` hacían `render_error_token(request, f'Error al procesar la solicitud: {str(e)}')`: exactamente el CWE-209 que cierra esta sección, en una página que ve quien llega desde un correo, posiblemente sin sesión iniciada. El texto de una excepción de base de datos lleva fragmentos de SQL, nombres de tabla y de columna o el nombre de la restricción violada.

Ahora esos cuatro puntos llaman a `render_error_token_inesperado(request, e, contexto)` (`core/utils/error_token.py`), que registra la traza con `logger.exception` —el filtro le adjunta el mismo identificador que se muestra en pantalla— y devuelve el mensaje genérico `MENSAJE_INESPERADO` con un **500**. El docstring del módulo fija la regla: en `mensaje` solo va texto redactado por nosotros, nunca `str(excepcion)`.

Segundo arreglo en las mismas vistas: devolvían **200 OK** en todos los fallos, lo que los hacía invisibles para cualquier alarma que vigile códigos de error. Ahora **403** para un token inválido o expirado, **409** para un rechazo por reglas de negocio o por falta de supervisor, y **500** para lo inesperado. `render_error_token` conserva `status=200` por defecto solo por compatibilidad; hay un test que exige `status=` explícito en todas las llamadas de esas vistas.

La tercera vía: las APIs JSON. Mismo patrón, tercer frente. Nueve `except Exception` de vistas API devolvían el error crudo dentro del propio JSON:

```
except Exception as e:
    return JsonResponse({'error': f'Error al obtener festivos: {str(e)}'},
                        status=500)
```

Son respuestas normales de una vista, no handlers, así que tampoco las tocó el barrido inicial. `str(e)` de MySQL puede ser `(1054, "Unknown column 'turnos_diaespecial.descripcion' in 'field list'")` —nombres reales de tabla y columna— o `(2003, "Can't connect to MySQL server on 'swalp-prod.xxxx.rds.amazonaws.com'")`, que **expone el endpoint de RDS**: el caso más serio de los tres frentes por lo que revela, aunque no el más expuesto por quién lo alcanza.

Atenuante respecto a la fuga original del 404: las nueve **exigen sesión iniciada**. Cinco las alcanza cualquier empleado autenticado; tres solo un supervisor —las dos de `turnos/api/views/calculo_automatico.py`, con `SupervisorApiRequiredMixin (:37,83)`, y la de `solicitudes/views/gestion_solicitudes.py`, con `AdminRequiredMixin (:126)`—. Ninguna es anónima.

Punto corregido	Vista	Quién llega
<code>turnos/api/views/dias_especiales.py:186-188</code>	<code>DiasFestivosView (:11)</code>	empleado autenticado
<code>turnos/api/views/dias_especiales.py:263-265</code>	<code>DiasTemporadaView (:191)</code>	empleado autenticado
<code>turnos/api/views/dias_especiales.py:352-354</code>	<code>DiasEspecialesPorTipoView (:268)</code>	empleado autenticado
<code>turnos/api/views/calculo_automatico.py:78-80</code>	<code>CalcularMantenimientoAutomaticoView (:37)</code>	supervisor
<code>turnos/api/views/calculo_automatico.py:130-132</code>	<code>CalcularFestivosAutomaticoView (:83)</code>	supervisor
<code>turnos/api/views/turnos_mes.py:22-24</code>	turnos AM/PM de un día	empleado autenticado

<code>solicitudes/views/api_turno_jornada.py:553-555</code>	cálculo de jornadas de un rango	empleado autenticado
<code>solicitudes/views/gestion_solicitudes.py:178-185</code>	reenvío de notificación desde Gestión	supervisor / staff

Los siete primeros llaman ahora a `json_error_inesperado(request, e, mensaje)` (`core/utils/json_responses.py:68-102`), hermano de `render_error_token_inesperado`: registra con `logger.exception (:99)` y devuelve **500** con el mensaje propio del endpoint más `extra: {request_id} (:101-102)`, el mismo código que el shim de `fetch` pinta en el aviso. El octavo no es JSON —pinta un `messages.error` en la pantalla de Gestión— y compone el código de referencia en el propio texto (`solicitudes/views/gestion_solicitudes.py:182-185`).

Decisión de diseño: mensaje específico por endpoint, no uno genérico único. Al contrario que la página del token, que usa `MENSAJE_INESPERADO`, aquí cada llamada pasa su propia frase ("No pudimos cargar los festivos. Inténtalo de nuevo.", `dias_especiales.py:188`; "No pudimos calcular los días de mantenimiento. Inténtalo de nuevo.", `calculo_automatico.py:80`). Un genérico único habría sido más barato de mantener, pero degrada la experiencia más de lo que exige la seguridad: el usuario pierde toda pista de **qué** falló. La regla que sí es dura la fija el docstring del helper (`core/utils/json_responses.py:96`): en `mensaje` solo va texto redactado por nosotros.

Hallazgo: seis de los nueve no dejaban traza. El arreglo no solo tapa la fuga, añade observabilidad que no existía. Tres no registraban absolutamente nada (`dias_especiales.py:186` y `:263`, `gestion_solicitudes.py:178`) y tres usaban `logger.error(f"...{e}")` sin `exc_info`, así que guardaban el mensaje pero **no el traceback**. Solo dos estaban bien. Se estaba en lo peor de ambos mundos: el detalle se le enseñaba a quien no le sirve y no se le guardaba a quien lo necesita.

La novena, y la peor: un depurador que se quedó puesto. `MisTurnosPorMesView` (`turnos/api/views/turnos_mes.py`) no solo interpolaba `str(e)`: devolvía en el propio JSON `'traceback': error_trace if request.user.is_staff else None`. Es decir, **publicaba el traceback completo** —rutas de fichero, líneas de código y nombres de variables— a cualquier usuario `is_staff`, que aquí incluye a los supervisores. No era un descuido como las otras ocho: era una puerta abierta a propósito para depurar y nunca cerrada. La acompañaban dos `print()`, sin nivel y sin identificador de petición, de modo que en CloudWatch quedaban sueltos y sin poder cruzarlos.

El test de regresión tenía un agujero, y por eso se escapó. La primera versión comparaba **línea a línea**, así que un `JsonResponse(` en una línea y el `str(e)` en la siguiente pasaban limpios. Ahora analiza el **bloque completo** del `except`, ignora los comentarios con `tokenize` —los propios comentarios que explican las fugas ya corregidas citan el patrón y se daban por reincidencias— y añade `test_no_se_publica_ningun_traceback`, que prohíbe que un

`traceback` acabe en una respuesta aunque esté detrás de un `if request.user.is_staff`. La corrección se verificó ejecutando ambas versiones contra el código con la fuga: la antigua no la detecta, la nueva sí.

*Regla que deja este episodio: **una barrera de regresión que no se prueba contra el fallo que dice prevenir no es una barrera.** Antes de dar por buena una de estas comprobaciones, ejecútala contra el código defectuoso y comprueba que falla.*

Tests de regresión. `core/tests/test_paginas_error.py`, 19 tests;
`core/tests/test_json_error_inesperado.py`, 16 tests sobre las APIs JSON (que ni el endpoint de RDS ni los nombres de tabla y columna lleguen al cliente, que el mensaje propio sí, que la traza quede en el log y que el código de referencia viaje en `extra`), uno de ellos parametrizado sobre los cinco módulos —lista `MODULOS` (:25)— que recorre el fichero con expresión regular y **falla si reaparece `str(e)` dentro de un `except Exception`** (:114-115); más `core/tests/test_error_token.py`, 8 tests sobre la página de los enlaces de correo (que no se filtre la excepción, que la traza sí quede en el log, los estados HTTP y que no reaparezca ninguna interpolación de `str(e)`) (`pytest core/tests/test_paginas_error.py --collect-only -q`), en tres bloques: que no se filtre nada de la lista `FUGAS` (:16-23), que el código de referencia funcione y sea del servidor (:60-91), que `base.html` defina el stub **antes** de cargar el fichero y este antes que jQuery, y que las cinco plantillas rendericen sin `request` y sin cargar `http(s)://` (:94-128).

10. Configuración

Un único `config/settings.py` gobernado por `ENVIRONMENT`; no hay `settings/local.py` (`config/settings.py`:26-27). Se lee con `django-environ` desde `.env` (`config/settings.py`:10,20,24); el porqué está en [ADR 004](#) (`./03-arquitectura/adr/004-variables-de-entorno-django-environ.md`).

Ningún valor real aparece en este manual. Los valores viven en `.env`, que no se versiona; la plantilla es `.env.example`.

Variable	Ob lig.	Tipo	Por defecto	Qué se rompe si falta	Dónde se lee
<code>ENVIRONMENT</code>	sí	<code>str</code> (<code>development</code> \	<code>production</code>)	—	No arranca: gobierna todo el archivo

SECRET_KEY	sí	str	—	Improperly Configured al importar settings	config/settings.py:32
DEBUG	sí	bool	—	Ídem	config/settings.py:33
ALLOWED_HOSTS	no	lista	['127.0.0.1', 'localhost']	400 en cualquier otro host	config/settings.py:34
CSRF_TRUSTED_ORIGINS	no	lista	[]	POST rechazados tras un proxy con otro origen	config/settings.py:38
DB_NAME	sí	str	—	No conecta a la base	config/settings.py:165
DB_USER	sí	str	—	Ídem	config/settings.py:166
DB_PASSWORD	sí	str	—	Ídem	config/settings.py:167
DB_HOST	no	str	localhost	Apunta a la base equivocada	config/settings.py:168
DB_PORT	no	str	3306	Ídem	config/settings.py:169
DB_CONN_MAX_AGE	no	int	60 en prod, 0 en dev	Reconexión en cada petición (latencia)	config/settings.py:137
DB_SSL_CA	no	str (ruta)	''	Sin TLS contra RDS	config/settings.py:152
TEST_DB_NAME	no	str	test_<DB_NAME>	Dos corridas de tests comparten base y se pisan	config/settings.py:186
EMAIL_HOST	no	str	smtp.gmail.com	Correo al servidor equivocado	config/settings.py:238
EMAIL_PORT	no	int	587	Ídem	config/settings.py:239
EMAIL_USE_TLS	no	bool	True	Credenciales en claro	config/settings.py:240

EMAIL_HOST_USER	sí	str	—	Improperly Configured	config/settings.py:241
EMAIL_HOST_PASSWORD	sí	str	—	Ídem	config/settings.py:242
DEFAULT_FROM_EMAIL	sí	str	—	Ídem	config/settings.py:243
EMAIL_TIMEOUT	no	int (s)	10	Un SMTP colgado congela la petición	config/settings.py:246
EMAIL_SEND_ASYNC	no	bool	True en prod	En dev y tests el envío es síncrono y determinista	config/settings.py:251
SITE_URL	no	str	http://127.0.0.1:8000	Los enlaces de aprobación del correo apuntan a localhost	config/settings.py:253
APPROVAL_LINK_MAX_AGE_DAYS	no	int (días)	30	Nada al arrancar: sin la variable rigen 30 días. Bajarla acorta la vida de los enlaces del correo ya enviados; subirla amplía la ventana en que un enlace filtrado sigue sirviendo	config/settings.py:262, leída en solicitudes/services/tokens_aprobacion.py:51
CACHE_URL	no	str	' ' (LocMem)	Con más de un worker, la invalidación de "Mis Turnos" solo limpia un proceso	config/settings.py:283

CORS_ALLOWED_ORIGINS	no	lista	localhost:8000	Peticiones cruzadas bloqueadas	config/settings.py:321
SECURE_HTTPS	no	bool	True en prod	Sin redirección a HTTPS, o bucle de redirección si no hay proxy	config/settings.py:437
LOG_DIR	no	str (ruta)	<BASE_DIR>/logs	Nada: si no es escribible se renuncia al fichero y todo sale por stdout (§ 13.5)	config/settings.py:502

Valores de `CACHE\_URL` (config/settings.py:283-301): vacío → LocMemCache, solo desarrollo con un proceso; redis://host:6379/1 o rediss://... → backend Redis nativo de Django 5, opción recomendada en AWS; db://cache_appturnos → tabla en la propia MySQL, que exige python manage.py createcachetable una vez.

Diferencias dev / test / producción

Aspecto	development	test	producción
ENVIRONMENT	development	el del .env	production
Caché	LocMem	LocMem	Redis o tabla, compartida
EMAIL_SEND_ASYNC	False	False	True
django-axes	activo	desactivado (config/settings.py:336-341)	activo
SECURE_HTTPS	False	False	True
Base de datos	DB_NAME	TEST_DB_NAME (config/settings.py:186)	DB_NAME
Test runner	—	core.test_runner.NoInputDiscoverRunner (config/settings.py:193)	—

debug_toolbar	solo con <code>DEBUG=True</code> (<code>config/urls.py:40-44</code>)	no	no
----------------------	--	----	----

En Docker, ambos ficheros compose fijan `ENVIRONMENT=production` con `SECURE_HTTPS=False`, para poder navegar en HTTP local (§ 2.7).

11. Dependencias

11.1 Producción (requirements.txt)

Paquete	Versión	Para qué	Riesgo al actualizar
Django	5.2.16	Framework	Alto: subir de mayor exige revisar, entre otras cosas, <code>CheckConstraint(condition=...)</code> , que cambió de nombre respecto a <code>check=</code> — el proyecto usa las dos formas (<code>turnos/models.py:91</code> vs <code>solicitudes/models.py:322</code>)
PyMySQL	1.1.2	Driver MySQL puro Python, instalado como <code>MySQLdb</code> si falta el nativo (<code>config/settings.py:115-119</code>)	Medio: <code>install_as_MySQLdb()</code> es sensible a la versión
django-environ	0.14.0	Lee <code>.env</code> y tipa las variables	Bajo
django-simple-history	3.8.0	Tabla <code>historical*</code> por modelo	Alto: cada subida suele traer migraciones nuevas en todos los modelos con historial, que aquí son casi todos
django-axes	7.0.1	Bloqueo por intentos fallidos de login	Medio: la rama 7.x cambió nombres de ajustes de bloqueo
django-csp	4.0	Cabeceras CSP	Medio: la 4.0 introdujo el formato de diccionario <code>CONTENT_SECURITY_POLICY</code> que usa este proyecto
django-cors-headers	4.9.0	CORS	Bajo

django-widget-tweaks	1.5.0	Retoques de widgets en plantillas	Bajo
whitenoise	6.8.2	Sirve los estáticos desde el propio proceso, sin Nginx en el contenedor	Bajo
gunicorn	23.0.0	Servidor WSGI de producción	Bajo
redis	5.2.1	Solo el cliente. El backend de caché es el nativo de Django 5; no hace falta django-redis (comentario en requirements.txt)	Bajo
openpyxl	3.1.5	Exportación de reportes a Excel	Bajo
cryptography , cffi , pycparser	46.0.3 / 2.0.0 / 2.23	Transitivas de TLS	Medio: cryptography exige ruedas binarias compatibles con el intérprete
python-dateutil	2.9.0.post 0	Aritmética de fechas	Bajo
tzdata	2025.2	Zonas horarias en sistemas sin base tz	Bajo pero necesario : USE_TZ=True con America/Bogota
asgiref , sqlparse , six	3.11.0 / 0.5.3 / 1.17.0	Transitivas de Django	Bajo

Todas las versiones están **fijadas con `==`**. Es deliberado: sin lockfile, el pin exacto es lo único que hace reproducible un **pip install**.

11.2 Desarrollo (requirements-dev.txt)

Hereda producción con **-r requirements.txt** y añade cinco herramientas: **django-debug-toolbar** 6.2.0, **pytest** 8.3.4, **pytest-django** 4.9.0, **pytest-xdist** 3.8.0 y **pytest-cov** 7.1.0.

pytest-xdist no está solo por velocidad: da a cada worker su propia base de test con sufijo **_gwN**, lo que elimina de raíz que dos corridas simultáneas se pisen la misma base **test_*** (comentario de **requirements-dev.txt** sobre **pytest-xdist**).

11.3 Frontend y externas

- **Sin paso de build.** AdminLTE 3.2 y JavaScript vanilla servidos como estáticos (**docs/03-arquitectura/TECNOLOGIAS_FRONTEND.md**).
- **Autohospedados en `static/plugins/`:** flatpickr, chart.js, sweetalert2, fullcalendar,

ionicons y Font Awesome 5.15.4 (`config/settings.py:349-352,394-396`).

- **Lo único externo que queda:** Google Fonts (`fonts.googleapis.com`, `fonts.gstatic.com`)

en `base.html` y el login. Es lo único que la política CSP objetivo sigue permitiendo (`config/settings.py:396`).

- **SMTP / SES.** Se configura por variables de entorno (§ 10); el envío pasa siempre por `EmailOutbox`.

- **AWS.** RDS o Aurora para MySQL, ElastiCache opcional para la caché; ver § 13 y

[05-referencia/deployment/](#) (`./05-referencia/deployment/`).

No se encontró integración activa con Google Apps Script en el código de `AppTurnosExplora/`; ver § 18.

12. Recetas (how-to)

12.1 Añadir un tipo de solicitud

Guía completa: [04-guias/GUIA_AGREGAR_NUEVO_TIPO_SOLICITUD.md](#) (`./04-guias/GUIA_AGREGAR_NUEVO_TIPO_SOLICITUD.md`). El recorrido, resumido:

82. Añade el nombre a `TipoSolicitud` en `core/constants.py` y su equivalente en

`TipoCambioTurno` si escribe turnos; actualiza `MAPA_SOLICITUD_A_TURNO`.

83. Amplía `TipoCambioTurno.TODOS` y migra: la `CheckConstraint turno_tipo_cambio_valido` (`turnos/models.py:90-94`) rechazará el valor nuevo hasta entonces.

84. Crea la fila de `TipoSolicitudCambio` con ese nombre y su `codigo_estrategia`.

85. Crea `services/strategies/<tipo>_strategy.py` implementando el contrato de

`SolicitudStrategy` (`services/strategies/base_strategy.py`): `validar_solicitud`, `crear_solicitud`, `aplicar_cambios`, `revalidar_para_aprobar` (:84) y `get_empleados_disponibles`.

86. Regístrala en `SolicitudFactory` (`services/solicitud_factory.py:123-187`). Si cae en la estrategia por defecto verás un `warning` en el log (:182-187).

87. Si genera o revierte turnos, crea `services/<tipo>_aplicacion_service.py` con snapshot previo y resultante, y engánchalo en `_revertir_por_tipo` (`use_cases/cancelar_solicitud.py:474`).

88. Ejecuta `python manage.py actualizar_codigos_estrategia`.

89. Escribe el test **antes** de tocar el frontend.

12.2 Añadir un campo a un modelo

```
python manage.py makemigrations <app>
python manage.py migrate
```

Con `django-simple-history` la migración toca **dos** tablas: la del modelo y su `historical*`. Si el campo es obligatorio, dale `default` o hazlo `null=True`: la tabla histórica ya tiene filas y no puede quedarse sin valor. Si el campo participa en un invariante, exprésalo como `constraints` en `Meta` y no solo en `clean()`: `clean()` no corre en `save()` (patrón 26 de `PROTECTION_PATTERNS.md`, y por eso `Turno` valida `tipo_cambio` con una `CheckConstraint`, `turnos/models.py:90-94`).

12.3 Correr los tests

```
cd AppTurnosExplora
pytest                    # toda la suite
pytest -n 4               # en paralelo – unos 3 minutos
pytest solicitudes/tests/test_politica_temporada.py -v
pytest --cov=solicitudes --cov-report=term-missing
```

`pytest.ini` fija `DJANGO_SETTINGS_MODULE=config.settings` y los `testpaths` (`pytest.ini:2,11-18`). Ver § 14.

12.4 Depurar un fallo de validación

90. Identifica el tipo: `SolicitudCambio.tipo_cambio.nombre` frente a `TipoSolicitud` de

`core/constants.py`. Si estás filtrando `Turno.tipo_cambio`, el vocabulario es **otro** (`TipoCambioTurno`); confundirlos no da error, da un filtro que no casa con nada (`core/constants.py`, docstring del módulo).

91. Localiza la estrategia con `SolicitudFactory.get_strategy`

(`services/solicitud_factory.py:123`). Si cayó en la estrategia por defecto hay un `warning` en el log (`:182-187`): ahí está el problema.

92. Los chequeos transversales fallan **antes** que la estrategia. Mira el `code` de la

respuesta JSON: `cierre_semanal`, `sancionado`, `sancionado_receptor` y `duplicate_request` señalan al orquestador, no al tipo.

93. Si falla al **aprobar** y no al crear, es la re-validación (P5): el estado del día cambió entre medias.

94. Si el desplegable ofrece a alguien que el envío rechaza, es el patrón 37: selector y validación miran fechas distintas (`solicitudes/views/api_fin_semana.py:258-263`).

12.5 Regenerar la documentación

```
python .claude/skills/project-documentation-master/scripts/md_to_pdf.py \
  AppTurnosExplora/docs/manual_tecnico.md \
```

```
AppTurnosExplora/docs/pdf/Manual_Tecnico.pdf \
--titulo "Manual Tecnico" --subtitulo "AppTurnos / SWALP"
```

Requiere Microsoft Word (usa COM). El PDF **nunca** se edita a mano: se regenera desde el Markdown, que es la fuente de verdad.

12.6 Desplegar

Ver § 13 y los checklists de [05-referencia/deployment/](#) (./05-referencia/deployment/).

13. Despliegue

13.1 Topología

```
flowchart TB
    USER["Navegador<br/>explorador / supervisor"]
    subgraph AWS["AWS"]
        ALB["ALB – termina TLS<br/>health check /health/"]
        subgraph COMPUTE["Computo (EC2 o Fargate)"]
            GUNI["Gunicorn 3 workers<br/>+ WhiteNoise (estaticos)"]
            CRONJ["cron:<br/>procesar_email_outbox"]
        end
        RDS["RDS / Aurora MySQL 8.0<br/>TLS con DB_SSL_CA"]
        CACHEBOX["ElastiCache Redis<br/>o tabla cache_appturnos"]
        LOGS["CloudWatch Logs"]
    end
    USER --> ALB
    ALB --> GUNI
    GUNI --> RDS
    GUNI --> CACHEBOX
    GUNI --> LOGS
    CRONJ --> RDS
    CRONJ --> SES
    SES --> USER
```

Qué muestra. Un solo proceso de aplicación detrás de un balanceador. El ALB termina el TLS y reenvía por HTTP con `X-Forwarded-Proto`; sin `SECURE_PROXY_SSL_HEADER`, `SECURE_SSL_REDIRECT` provocaría un bucle infinito de redirecciones ([config/settings.py:439-441](#)). Los estáticos los sirve WhiteNoise dentro del propio proceso ([config/settings.py:73](#)): no hay Nginx en el contenedor. Gunicorn corre con 3 workers, y de ahí que la caché **tenga que ser compartida** —Redis o tabla— y nunca `LocMemCache` ([config/settings.py:279-296](#)). El envío de correo no lo hace la petición web: lo hace un cron independiente que vacía `EmailOutbox`. La base es MySQL 8.0 gestionada, con TLS si `DB_SSL_CA` apunta al certificado ([config/settings.py:152](#)).

Dos variantes documentadas: EC2 + RDS ([MANUAL_DESPLIEGUE_EC2.md](#) (./05-referencia/deployment/MANUAL_DESPLIEGUE_EC2.md)), [arquitectura-aws-rds-recomendada.md](#) (./05-

referencia/deployment/arquitectura-aws-rds-recomendada.md)) y Fargate
([CHECKLIST_DESPLIEGUE_FARGATE.md](#) (./05-referencia/deployment/CHECKLIST_DESPLIEGUE_FARGATE.md)).

13.2 Pipeline y pasos

No hay CI/CD automatizado en el repositorio. El despliegue es manual y sigue esta secuencia:

95. Construir la imagen desde `AppTurnosExplora/Dockerfile` (Python 3.12).
96. Publicarla en el registro.
97. Aplicar migraciones: `python manage.py migrate`.
98. `python manage.py collectstatic --noinput` — WhiteNoise sirve desde `STATIC_ROOT` (`config/settings.py:219`).
99. Arrancar Gunicorn con 3 workers.
100. Verificar `/health/` y `/health/ready/`.

Ojo: `docker-compose.hostdb.yml` **sobrescribe** el `command` para arrancar solo Gunicorn sin `migrate` (§ 2.7). En producción el `migrate` es un paso explícito, no implícito.

13.3 Checklist previo

- [] `python manage.py check --deploy` sin errores críticos. Conviene convertirlo en `**puerta del`

`pipeline**` con `--fail-level WARNING`, para que un `DEBUG=True` colado no llegue a desplegarse.

- [] `ENVIRONMENT=production` y `DEBUG=False`.
- [] `SECRET_KEY` distinta de la de desarrollo. **Pendiente real hoy:** con

`DEBUG=False SECURE_HTTPS=True python manage.py check --deploy` el único aviso que queda es `security.W009` — la clave sigue siendo la autogenerada con prefijo `django-insecure-`, y además estuvo expuesta en pantallas de debug. Con ella se firman las cookies de sesión: quien la tenga puede falsificar la sesión de cualquier usuario, incluido un superusuario. Generar la nueva con `python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"` y guardarla en Secrets Manager. No se ha rotado en el repositorio a propósito: invalida todas las sesiones activas, así que es una decisión de despliegue.

- [] `SECRET_KEY` **idéntica en todas las instancias:** una sola entrada en Secrets Manager o SSM, nunca un valor generado por tarea. **El procedimiento documentado ya lo garantiza** (§ 9.2); esto se marca comprobando en la consola que `swalp/SECRET_KEY` es un único secreto y que el task definition lo inyecta en `secrets:`, no en `environment:`. Firma los tokens de aprobación por correo, y si cada instancia usa la suya los enlaces fallan de forma intermitente según a cuál encamine el ALB (`solicitudes/services/tokens_aprobacion.py:59-62,84`). Rotarla invalida los enlaces ya enviados: avisa antes.

- [] `APPROVAL_LINK_MAX_AGE_DAYS` revisada si el plazo de respuesta real no cabe en 30 días

([config/settings.py:262](#)).

- [] [ALLOWED_HOSTS](#) y [CSRF_TRUSTED_ORIGINS](#) con el dominio real.
- [] [SITE_URL](#) con el dominio real: si no, los enlaces de aprobación del correo apuntan a

[127.0.0.1](#) ([config/settings.py:253](#)).

- [] [CACHE_URL](#) apuntando a Redis o a tabla. **Nunca vacío con más de un worker.**
- [] Si es tabla: `python manage.py createcachetable` ejecutado una vez.
- [] [DB_SSL_CA](#) con el bundle de RDS.
- [] [SECURE_HTTPS=True](#) y el ALB reenviando [X-Forwarded-Proto](#).
- [] Cron de [procesar_email_outbox](#) activo.
- [] **Alarma del cierre semanal creada** (§ 13.6): metric filter sobre

[CIERRE SEMANAL INOPERATIVO](#) + alarma con umbral 1 hacia SNS. Sin esto, el cierre puede quedar desactivado de hecho sin que nadie se entere: la comprobación falla **abierta** a propósito y su única señal es una línea de log.

- [] Calendario del año cargado (§ 2.5): festivos, temporada, mantenimiento, alternancia y descansos de semana.
- [] Los seis [TipoSolicitudCambio](#) creados con los nombres exactos de [core/constants.py](#).
- [] Los roles [Supervisor](#) y [Explorador](#) y las jornadas [AM](#) y [PM](#) existentes.

Checklists completos: [CHECKLIST_DESPLIEGUE_AWS_RDS.md](#) (./05-

[referencia/deployment/CHECKLIST_DESPLIEGUE_AWS_RDS.md](#)) y [CONFIGURACION_PRODUCCION.md](#) (./05-
[referencia/deployment/CONFIGURACION_PRODUCCION.md](#)).

13.4 Rollback

Volver a la imagen anterior es inmediato. **Las migraciones no:** varias añaden constraints que no se pueden revertir sin perder la garantía (§ 5.3). Regla práctica: si el despliegue incluyó migraciones que crean [UniqueConstraint](#) o [CheckConstraint](#), el rollback de código es seguro pero el de esquema exige revisión manual. Antes de cualquier despliegue con migraciones, snapshot de RDS.

13.5 Observabilidad

Qué	Dónde se mira
Salud del proceso	GET /health/ — no toca la base
Salud de la base	GET /health/ready/
Errores de aplicación	logger.exception en vistas y servicios → stdout → CloudWatch
Traza de una petición concreta	El código de referencia que el usuario ve en la página de error (§ 9.5). Filtrarlo en CloudWatch devuelve todas sus líneas

Cierre semanal caído	Alarma sobre CIERRE SEMANAL INOPERATIVO (ver § 13.6). Buscarlo a mano no basta: el fallo es silencioso
Tipo de solicitud sin estrategia	warning de la factory (<code>solicitudes/services/solicitud_factory.py:182-187</code>)
Correos atascados	EmailOutbox con <code>estado='fallido'</code> — agotó los 5 intentos (<code>solicitudes/models.py:69,77</code>)
Solicitudes atascadas	Pantalla <code>/solicitudes/gestion-solicitudes/</code>
Auditoría de cambios	Tablas historical* de django-simple-history

Configuración de LOGGING

Definida en `config/settings.py:466-576`. Los errores **no** se le muestran al usuario (§ 9.5): salen por aquí, y solo por aquí. Dos destinos, cada uno por un motivo distinto:

Handler	Destino	Por qué
console	stdout	En ECS/Fargate el log driver awslogs recoge stdout y lo publica en CloudWatch Logs sin dependencias extra . Si el contenedor muere, el driver ya envió lo que había
file	<code>logs/appturnos.log</code> , RotatingFileHandler de 10 MB × 5 backups (~50 MB de techo)	Sobrevive a un corte de red con CloudWatch y permite un tail -f inmediato por SSH. El CloudWatch agent puede además vigilar el fichero si se quiere un segundo grupo con otra retención

Loggers configurados: `solicitudes` e `turnos` a **INFO**; `django.request` a **ERROR** —**aquí aterrizan los 500 con su traceback completo**, es el logger que sustituye a la pantalla de debug—; `django.security` a **WARNING** (fallos de CSRF, `Host` fuera de `ALLOWED_HOSTS`); y `core.errors`. La raíz, a **WARNING**.

El filtro ``request_id``. `core.errors.RequestIDFilter` inyecta el identificador en cada registro, y ambos formatters lo imprimen entre corchetes. Por eso buscar el código que el usuario reporta reconstruye la petición entera:

```
aws logs filter-log-events --log-group-name /swalp/app --filter-pattern
'"A3F91C2B7D01"'
```

Fuera de una petición el campo vale `-`.

El fichero puede no existir, y es intencionado. `LOG_DIR` se puede reapuntar por entorno —por ejemplo a un volumen montado, cuando el contenedor arranca con el sistema de ficheros en **solo lectura**, que es lo recomendable en Fargate—. Si el directorio no se puede crear o no es escribible, `LOG_A_FICHERO` queda en `False` y `_DESTINOS` se reduce a `['console']`: la aplicación **no se cae por no poder escribir un log**, simplemente renuncia al fichero y todo sale por stdout, que es de donde tira CloudWatch de todos modos.

*Retención y datos personales. Estos logs contienen nombres de empleado y detalles de solicitudes. La retención se configura **en el grupo de CloudWatch**, no en el fichero, y el acceso al grupo se restringe por IAM. `Logs/` está en `.gitignore`: no se versiona nunca.*

13.6 Alarma obligatoria: el cierre semanal falla ABIERTO

`ProcesarSolicitudOrchestrator.verificar_cierre` envuelve toda la comprobación del cierre en un `try/except Exception`. Si la verificación se rompe, **la solicitud se acepta sin validar el cierre** y solo queda un `logger.critical` (`solicitudes/services/solicitud_orchestrator.py:154-162`).

La decisión de fallar abierto es deliberada y está razonada en el docstring: un *fail-closed* dejaría a **todos** los exploradores sin poder enviar ninguna solicitud, mientras que dejar pasar alguna fuera de plazo el supervisor todavía puede rechazarla a mano. Romper el formulario a todo el mundo es peor que colar una solicitud.

Pero esa decisión solo es aceptable si alguien se entera. El comentario del código dice que se registra en CRITICAL "para que el fallo sea visible en alertas"; esa frase asume una alerta que **hay que crear en el despliegue**. Sin ella, el cierre semanal queda desactivado de hecho y nadie lo nota: no hay error visible ni formulario roto, simplemente empiezan a entrar solicitudes fuera de plazo una a una. Es un fallo silencioso y progresivo, el peor tipo.

Hoy el único destino de los logs es `console` → stdout (`config/settings.py`, bloque `LOGGING`), y `ADMINS` no está definido, así que **no existe ningún envío automático**.

Fargate / ECS (recomendado)

El `logConfiguration` ya manda stdout al grupo `/ecs/swalp`. Falta el filtro y la alarma:

```
# 1. Tema SNS al que suscribir a quien deba enterarse
aws sns create-topic --name swalp-alertas
aws sns subscribe --topic-arn <arn-del-tema> --protocol email --notification-
endpoint <tu-correo>

# 2. Metric filter: cuenta las apariciones del texto en el log
aws logs put-metric-filter \
  --log-group-name /ecs/swalp \
  --filter-name swalp-cierre-inoperativo \
  --filter-pattern '"CIERRE SEMANAL INOPERATIVO"' \
  --metric-transformations \
```

```
metricName=CierreSemanalInoperativo,metricNamespace=SWALP,metricValue=1,defaultValue=0

# 3. Alarma: una sola aparición ya es motivo de aviso
aws cloudwatch put-metric-alarm \
  --alarm-name swalp-cierre-semanal-inoperativo \
  --namespace SWALP --metric-name CierreSemanalInoperativo \
  --statistic Sum --period 300 --evaluation-periods 1 \
  --threshold 1 --comparison-operator GreaterThanOrEqualToThreshold \
  --treat-missing-data notBreaching \
  --alarm-actions <arn-del-tema>
```

El umbral es **1**: una sola aparición significa que el cierre ya no está protegiendo nada. El **Sum** sobre 5 minutos además distingue un fallo puntual de una avería continua (doscientas apariciones en una hora no es lo mismo que una suelta).

Conviene una segunda alarma con el mismo patrón sobre **?ERROR ?CRITICAL** para no depender de haber previsto cada texto concreto.

EC2 (instancia única)

No hay CloudWatch por defecto: los logs de Unicorn van a **journald**. Dos opciones:

- Instalar el **CloudWatch agent** apuntando al journal de la unidad y aplicar el mismo filtro.
- Sin agente, un cron diario que avise:

```
journalctl -u appturnosex --since "24 hours ago" | grep -q "CIERRE SEMANAL
INOPERATIVO" \
  && echo "Cierre semanal inoperativo en las ultimas 24h" | mail -s "SWALP:
alerta" <tu-correo>
```

Qué hacer cuando la alarma suena (runbook)

Que suene significa **dos** cosas, y la segunda se olvida: la comprobación está rota **y**, mientras lo estuvo, entraron solicitudes sin validar el cierre. Arreglar solo lo primero deja solicitudes coladas para un fin de semana ya cerrado que nadie va a mirar, porque no dieron error.

Prioridad: no es una caída. La aplicación sigue funcionando y el supervisor conserva la última palabra sobre cada solicitud. Se atiende en horario laboral. Lo que no se puede es ignorar: cada día que pasa son más solicitudes que revisar hacia atrás.

101. **Causa.** El log lleva **exc_info=True**: la traza completa viaja en el propio mensaje de CloudWatch. No hay que reproducir nada. Causas típicas: base inaccesible o con *timeout*, dato mal cargado en **CierreSolicitudesConfig / CierreSemanaOverride**, festivos o mantenimientos sin cargar para el año, o un fallo introducido en el último despliegue.

102. **Ventana afectada.** Anotar la hora del **primer CRITICAL**: es el momento en que dejó de haber protección.

103. **Encontrar lo que se coló.** Esta consulta reutiliza el mismo servicio que hace la validación,

así que aplica exactamente el criterio que se saltó:

```
# python manage.py shell
from solicitudes.models import SolicitudCambio
from solicitudes.services.cierre_solicitudes_service import
CierreSolicitudesService as C
from django.utils.dateparse import parse_datetime

desde = parse_datetime('2026-08-15 03:12:00+00:00') # hora del primer CRITICAL
for s in SolicitudCambio.objects.filter(fecha_solicitud__gte=desde,
estado='pendiente'):
    f, msg = C.validar_fechas([s.fecha_cambio_turno])
    if f:
        print(s.id, s.explorador_solicitante, s.fecha_cambio_turno, '->', msg)
```

Lo que aparezca **no debería haber entrado**; el supervisor lo rechaza desde `/solicitudes/gestion-solicitudes/`.

⚠ Cubre las solicitudes de fecha única (`fecha_cambio_turno`). Los tipos **permanentes** guardan su rango en tablas relacionadas (`CambioPermanenteDetalle`) y hay que revisarlos aparte.

104. **Arreglar y confirmar de verdad.** Cuando la alarma vuelva a OK, crear una solicitud de prueba

para una fecha **dentro** de la ventana cerrada: debe rechazarse con "Cierre de solicitudes activo..." y código `cierre_semanal`. Que deje de haber **CRITICAL** no prueba que la validación funcione — prueba que dejó de ejecutarse la rama del `except`.

Por qué NO se resuelve con mail_admins

Django trae `AdminEmailHandler`, que enviaría el CRITICAL por correo con solo definir `ADMINS`. Se descarta a propósito: ese handler envía **de forma síncrona y dentro del hilo del request**, saltándose el patrón *outbox*. Es exactamente el bloqueo de ~20 s por handshake SMTP que el proyecto eliminó con `EMAIL_SEND_ASYNC` (§ 7.3). Además, un fallo de correo es una de las causas plausibles de la avería que se quiere avisar: no conviene que la alarma dependa del subsistema que puede estar roto. La alarma vive fuera de la aplicación.

14. Pruebas

14.1 Cómo se corren

```
cd AppTurnosExplora
pytest # suite completa
pytest -n 4 # en paralelo – unos 3 minutos
```

`pytest.ini` fija el módulo de settings, los patrones de descubrimiento `—test_*.py, Test*, test_*—` y `--strict-markers` (`pytest.ini:2-10`).

14.2 Cómo está partida la suite

testpath	Archivos	Qué cubre
<code>solicitudes/tests</code>	43	El dominio central: los seis tipos, validadores, aprobación, cancelación, reversión y temporada
<code>turnos/tests,</code> <code>empleados/tests,</code> <code>permisos/tests,</code> <code>core/tests,</code> <code>integration_tests</code>	33 entre todos	Calendario, personas, permisos, utilidades y flujos de punta a punta

La seguridad de los enlaces de correo tiene archivo propio:

`solicitudes/tests/test_tokens_aprobacion.py`, 25 tests (`pytest solicitudes/tests/test_tokens_aprobacion.py --collect-only -q`), que cubren firma, caducidad, aislamiento por sal, rol equivocado y suplantación.

Los `testpaths` están declarados en `pytest.ini:11-18`. `AppTurnosExplora/scripts/` **no** está en la lista: son utilidades manuales y `pytest` no las recoge.

14.3 Base de datos de test

Cuatro decisiones que evitan fallos fantasma:

- `TEST_DB_NAME` da un nombre explícito a la base de test, por defecto `test_<DB_NAME>` (`config/settings.py:186`). Sin él, dos corridas simultáneas compartían base y producían fallos masivos sin relación con el código.
- `pytest-xdist` da a cada worker su propia base con sufijo `_gwN` (comentario de `requirements-dev.txt`). Por eso `-n 4` no solo va más rápido: aísla.
- `TEST_RUNNER = 'core.test_runner.NoInputDiscoverRunner'` evita que una base `test_*` huérfana —de una corrida que murió a medias— deje la ejecución esperando un "yes" que nadie puede escribir (`config/settings.py:190-194`).
- `django-axes` está desactivado bajo `pytest` (`config/settings.py:336-341`).

14.4 Cobertura y áreas sin cubrir

El inventario de tests está en [05-referencia/pruebas/INVENTARIO_TESTS.md](#) (`./05-referencia/pruebas/INVENTARIO_TESTS.md`).

Áreas con menos cobertura relativa, medidas por número de archivos de test frente al peso del módulo: la app `permisos/` (dos modelos, PDH y permisos especiales) y las vistas de `empleados/` (salas, competencias, restricciones, indicadores). El porcentaje exacto de cobertura no se midió en esta pasada: ver § 18.

15. Decisiones técnicas (ADR)

ADR	Título	En una línea
001 (./03-arquitectura/adr/001-service-layer-y-orchestrator.md)	Service Layer y patrón Orchestrator	La lógica sale de las vistas a servicios, con un orquestador que encadena los chequeos transversales
002 (./03-arquitectura/adr/002-fsm-sin-libreria-externa.md)	Máquina de estados sin librería externa (django-fsm)	Las transiciones son pocas y fijas; la librería añadía dependencia sin cerrar el hueco real
003 (./03-arquitectura/adr/003-on-commit-para-notificaciones.md)	<code>transaction.on_commit()</code> para desacoplar notificaciones	Un fallo de correo no revierte la operación de negocio. Superado en parte por el outbox (<code>solicitudes/models.py:44-64</code>), que además garantiza la entrega
004 (./03-arquitectura/adr/004-variables-de-entorno-django-environ.md)	Variables de entorno con django-environ	Configuración fuera del código, un único <code>settings.py</code>
005 (./03-arquitectura/adr/005-pendientes-aws.md)	Pendientes para despliegue en AWS	Lista de trabajo de infraestructura
006 (./03-arquitectura/adr/006-tokens-firmados-para-aprobacion-por-correo.md)	<code>django.core.signing</code> para los tokens de aprobación por correo	Una credencial que actúa sin sesión se firma en un solo sitio con <code>SECRET_KEY</code> y caduca. Recoge por qué se descartaron el HMAC propio, la tabla de tokens gastados y la caché

| [007](#) (./03-arquitectura/adr/007-paginas-de-error-propias-y-request-id.md) | Páginas de error propias e identificador de petición | Las pantallas técnicas de Django filtraban el URLconf completo y el traceback. Se sustituyen por cinco plantillas autocontenidas y se traza cada petición con un código opaco que el usuario puede reportar. Ver § 9.5 |

Los cuatro primeros están fechados en 2026-06 y marcados **Implementado**; el 005 no lleva cabecera de estado y es una lista de pendientes, no una decisión cerrada.

Regla: una decisión técnica nueva se registra como ADR en `docs/03-arquitectura/adr/`, no como párrafo suelto en este manual. Decisiones tomadas y documentadas fuera del formato ADR que deberían tener el suyo: el patrón `outbox`, la `UniqueConstraint` con columna discriminante `activo_key`, y `core/constants.py` como fuente única de vocabularios. Ver § 18.

16. Zonas frágiles y bugs conocidos

16.1 El catálogo de protecciones

`PROTECTION_PATTERNS.md` (raíz del repositorio) recoge **37 patrones** aplicados, cada uno con su motivo y el caso real que lo provocó. Es lectura obligatoria antes de tocar cualquier flujo de escritura. Los que más restringen lo que puedes hacer:

#	Patrón	Qué te obliga a hacer
2	Select-for-update + transacción	Bloqueo pesimista sobre las filas que vas a modificar
3	Idempotent guards / snapshot-once	El snapshot se captura una sola vez , antes de aplicar; re-aplicar no lo reescribe
21	Deuda idempotente por (explorador, fecha)	Reintentar no duplica deuda
22	Reconciliación completa	Cubrir todos los modelos de detalle, no solo el del tipo que tocaste
25	Guardias que fallan cerrado	Ante la duda, bloquear
26	Invariantes estructurales en la base de datos	Un <code>clean()</code> no basta: no corre en <code>save()</code>
27	Una sola noción de "hoy": <code>timezone.localtime()</code>	Nunca <code>date.today()</code>
28	El turno real (L1) manda sobre la atribución de descanso (L2)	Validar contra el estado real, no contra la jornada base
29	Un hueco en L2 no da error: da un dato falso	Comprobar la ausencia explícitamente
30	Un snapshot solo vale si nadie tocó el día	De ahí <code>snapshot_turnos_resultantes</code> y la guardia de integridad
31	Un catálogo que gobierna permisos se compara EXACTO y se protege	Role y Jornada

32	Un hecho que se registra no se borra, y su fin previsto no es su fin real	SancionEmpleado
33	Al re-aplicar, el conjunto de días afectados se cierra antes de escribir	Incluir los días colaterales
35	La regla se valida sobre el campo que manda al aplicar	No sobre el que la declara
37	Un desplegable de candidatos filtra por las mismas fechas que valida el envío	Selector y validación se cambian en el mismo commit

El archivo tiene además una matriz de patrones por flujo ([PROTECTION_PATTERNS.md:1087](#)), un checklist para flujos nuevos ([:1541](#)) y una sección de **deuda conocida pendiente de decisión** ([:1731](#)) que no se debe aplicar todavía.

⚠ [PROTECTION_PATTERNS.md](#) vive **fuera** de [docs/](#), lo que rompe el "una sola fuente de verdad" del árbol documental. Ver § 18.

16.2 Zonas que se rompen fácil

Zona	Por qué es frágil	Qué hacer
Los tres campos llamados tipo_cambio	<code>SolicitudCambio.tipo_cambio</code> (FK), <code>Turno.tipo_cambio</code> y <code>TurnoArchivo.tipo_cambio</code> (texto) no comparten vocabulario . Mezclarlos no da error: da un filtro que no casa con ninguna fila y una validación que deja de aplicarse en silencio. Ya pasó, con <code>tipo_cambio_nombre='CT'</code> comparando un <code>codigo_estrategia</code> contra un <code>nombre</code> (<code>core/constants.py</code> , docstring)	Usar siempre <code>core/constants.py</code> y <code>MAPA_SOLICITUD_A_TURNO</code>
Reversión de una cancelación	Restaurar el snapshot pisa cambios ajenos si alguien tocó el día. Las dos guardias no ven permisos especiales, reprogramaciones ni ediciones del admin (<code>use_cases/cancelar_solicitud.py:289-300</code>)	No añadir rutas de escritura de turnos que salten los servicios de aplicación
Role y	Se buscan por nombre literal .	Los <code>NOMBRES_PROTEGIDOS</code> lo impiden; no los

Jornada	Renombrar "Supervisor" deja la operación sin supervisores; borrar AM/PM rompe el motor	ablandes (empleados/models.py:73-85,19-23)
Turnos duplicados	El invariante se sostenía por la disciplina de delete-then-create repetida en unos quince sitios	Ahora lo garantiza la base de datos (turnos/models.py:73-83). No desactives esa constraint
Selectores de compañero	Un desplegable que filtra por criterios distintos a los que valida el envío ofrece candidatos que el backend tumba	Patrón 37; ocurrió en 9c429e3 y en 4dd6cd6
Caché de "Mis Turnos"	Con LocMemCache y varios workers, la invalidación limpia un solo proceso	CACHE_URL obligatorio en producción (config/settings.py:279-296)
El shim de window.fetch	Está en el camino de todas las peticiones del frontend. Un .catch encadenado convertiría un fallo de red en promesa resuelta y los formularios dejarían de mostrar su aviso; devolver algo distinto de la Response original rompería a todos sus consumidores	Invariante: no encadenar .catch , no alterar argumentos ni respuesta, y mantener la lectura de la cabecera dentro del try/catch (static/js/utils/codigo-referencia.js:89-107). Candidato a patrón nuevo en PROTECTION_PATTERNS.md
Un except Exception nuevo en una vista API	Es la vía más fácil de reabrir CWE-209: JsonResponse({'error': str(e)}) parece inofensivo y puede acabar publicando el endpoint de RDS. No pasa por ningún handler de Django	Invariante: usar json_error_inesperado(request, e, mensaje) (core/utils/json_responses.py:68) con texto propio. El test parametrizado de core/tests/test_json_error_inesperado.py:114-115 vigila los cinco módulos ya saneados, pero no los que se añadan después : al crear un módulo API nuevo, súmalo a MODULOS (:25) . Ver § 9.5
Mapeo de errores por subcadena	CancelarSolicitudView decide el código HTTP inspeccionando el texto del mensaje ('más reciente' in msg)	Cambiar la redacción de un mensaje cambia el HTTP que ve el frontend (solicitudes/views/aprobacion_views.py:64-72)

16.3 Bugs y riesgos abiertos

#	Síntoma	Causa	Evidencia
B 3	No hay recuperación de contraseña. Tras 5 intentos fallidos el usuario queda	core/login/urls.py declara solo login y logout ;	core/login/urls.py:4-7 , config/settings.py:330-331

	bloqueado una hora y su única salida es un administrador	ninguna vista de reseteo está enrutada. <code>AXES_FAILURE_LIMIT=5,</code> <code>AXES_COOLOFF_TIME=1</code>	
B 4	El cierre semanal falla abierto. Si la comprobación revienta, la solicitud se acepta y solo queda un CRITICAL en el log	Decisión deliberada y documentada: romper el formulario a todos los exploradores es peor que colar una solicitud fuera de plazo, que el supervisor aún puede rechazar. El riesgo era que nadie vigilara ese log. Mitigado con procedimiento: la alarma sobre CIERRE SEMANAL INOPERATIVO es ahora un ítem obligatorio del despliegue (§ 13.6 y § 13.3, y los checklists de Fargate y EC2). El código no cambia: fallar abierto sigue siendo lo correcto; lo que faltaba era el aviso. Queda abierto hasta que la alarma exista de verdad en AWS — créala y pruébala	<code>solicitudes/services/solicitud_orchestrator.py:146-162</code> , § 13.6
B 5	Cobertura con dos compañeros: correo posiblemente irretirable. Si la segunda creación falla, <code>transaction.atomic()</code>	El outbox garantiza <i>como máximo una vez por clave</i> , no la retirada; el propio docstring de	<code>solicitudes/services/solicitud_orchestrator.py:262-280</code> , <code>solicitudes/models.py:61-64</code> . Alcance sin verificar: depende de si <code>crear_solicitud</code> encola dentro de la misma <code>atomic()</code> y de si el

	borra las dos filas, pero un correo ya enviado no se puede desenviar	EmailOutbox lo dice	worker ya reclamó la fila. Ver § 18
B 6	Comandos de diagnóstico con nombre de incidencia en el árbol de producción: test_verificar_dobla da_jeison, validar_fix_doblada_jeison, validar_dobladas_jun io, test_factory	Scripts puntuales que nunca se retiraron. Confunden a quien llega nuevo y ensucian manage.py help	solicitudes/management/commands/
B 7	Rutas comentadas en `urls.py`. El bloque <code>permisos-detalle/</code> está comentado con la nota "COMENTADO TEMPORALMENTE", igual que sus imports. Una de las líneas comentadas apunta además a <code>PermisoDetalleUpdate View</code> donde debería ir la de borrado	Deuda: o se restauran o se borran	solicitudes/urls.py, bloque de imports y bloque "ADMINISTRACIÓN"
B 8	CSP estricta en modo report-only indefinido. La política activa sigue permitiendo cuatro CDN que ninguna plantilla usa ya	Falta confirmar que no hay violaciones para promoverla	config/settings.py:349-361, 399-418

16.4 Resueltos que conviene recordar

#	Qué pasaba	Cómo se cerró
B1 (cerrado)	Los enlaces de aprobación por correo se firmaban con una clave escrita en el código , <code>b'secret_key_change_this'</code> , con el comentario "Cambiar en producción". Quien la conociera —está en el repositorio— podía fabricar un enlace válido para cualquier	Se centralizó todo en <code>solicitudes/services/tokens_aprobacion.py</code> , que firma con <code>django.core.signing</code> sobre <code>SECRET_KEY</code> . Los seis puntos delegan en él (<code>email_service.py:148,528</code> ; <code>views/aprobacion_email.py:140,199,251,303</code> ; <code>permisos/services.py:30,34</code>). Por eso el módulo está centralizado: no lo vuelvas a duplicar. El literal solo sobrevive dentro de

	solicitud y aprobarla sin sesión. La verificación estaba duplicada seis veces: una en <code>email_service.py</code>, cuatro en <code>views/aprobacion_email.py</code> y una en <code>permisos/services.py</code> (esta última, de otra app, ni siquiera estaba documentada). Cualquier arreglo había que hacerlo seis veces, y bastaba olvidar una para reabrir el agujero	<code>solicitudes/tests/test_tokens_aprobacion.py:68</code>, donde se usa a propósito para comprobar que un token firmado con la clave vieja ya no se acepta
B2 (cerrado)	Los tokens no caducaban ni se invalidaban tras usarse: eran deterministas sobre <code>(solicitud_id, empleado_id, tipo)</code>	La firma incluye marca de tiempo y se valida con <code>max_age = APPROVAL_LINK_MAX_AGE_DAYS</code> (<code>tokens_aprobacion.py:49-52,84</code>). El uso único lo garantiza <code>_ya_resuelto_para()</code> contra el estado en base de datos (<code>views/aprobacion_email.py:58-73</code>), no una lista en memoria. Ver § 9.2
B6 (cerrado)	Seis mensajes que ve el usuario tenían los acentos rotos por doble codificación UTF-8: en pantalla se leía literalmente <code>Token invÃ;lido o expirado</code> y <code>No se encontrÃ³ supervisor para esta solicitud</code>. Eran 31 secuencias en 5 archivos de <code>solicitudes/views/</code>	Corregidas todas. La comprobación de que no vuelven: <code>`grep -rn "Ãi\"</code>
B7 (cerrado)	La página de error del token no decía el plazo ni qué hacer, y un mismo mensaje cubre cuatro causas distintas (caducado, manipulado, ajeno, rol equivocado)	<code>templates/solicitudes/error_token.html</code> nombra ahora los <code>{{ dias_validez }}</code> días y remite a resolver desde la aplicación. El contexto se arma en <code>core/utls/error_token.py</code>, punto único por el que pasan las 15 llamadas de <code>solicitudes</code> y <code>permisos</code>: así ninguna vista puede olvidarse de pasar el dato y dejar la frase coja

Los dos primeros están generalizados como **patrón 38** de `PROTECTION_PATTERNS.md` (una credencial que actúa sin sesión se firma en un solo sitio, caduca y falla cerrada) y razonados en el [ADR 006](#) (`./03-arquitectura/adr/006-tokens-firmados-para-aprobacion-por-correo.md`).

17. Cambios técnicos relevantes

Changelog para desarrolladores, del más reciente al más antiguo. Cada línea corresponde a un commit real del repositorio.

Páginas de error propias y trazabilidad de la petición

Cambio en el árbol de trabajo, todavía sin commit al redactar esta entrada (rama `fix/cambio-descanso-temporada`). Detalle completo en § 9.5 y [ADR 007](#) (`./03-arquitectura/adr/007-paginas-de-error-propias-y-request-id.md`).

Cambio	Impacto
Nuevo <code>core/errors.py</code> : <code>RequestIDMiddleware</code> , <code>RequestIDFilter</code> , los cuatro handlers, <code>csrf_failure</code> y <code>previsualizar_error</code>	Con <code>DEBUG=True</code> cualquier URL inexistente devolvía el URLconf completo, con todos los endpoints de aprobación y rechazo (CWE-215)
Nuevas plantillas <code>templates/errors/_base_error.html</code> y <code>templates/{400,403,403_csrf,404,500}.html</code>	Autocontenidas: no heredan de <code>base.html</code> ni cargan recursos externos, así que funcionan con la base o los estáticos caídos
<code>handler400/403/404/500</code> en <code>config/urls.py</code> ; ruta de previsualización <code>/__error__/<codigo></code> solo bajo <code>DEBUG</code>	Las páginas se pueden revisar en desarrollo, donde Django nunca llega a usar los handlers
<code>core.errors.RequestIDMiddleware</code> como primer middleware y <code>CSRF_FAILURE_VIEW</code> propio	Cada petición lleva código opaco en <code>X-Request-ID</code> ; el motivo exacto del fallo CSRF deja de mostrarse al cliente
<code>LOGGING</code> reescrito: filtro <code>request_id</code> en ambos formatters, <code>RotatingFileHandler</code> junto a <code>stdout</code> , loggers <code>django.security</code> y <code>core.errors</code> , degradación a solo <code>stdout</code> si <code>LOG_DIR</code> no es escribible	El detalle del error va al equipo, no al usuario. Ver § 13.5
Se cierra la misma fuga en la página de los enlaces de correo: los cuatro <code>except Exception</code> de <code>views/aprobacion_email.py</code> dejan de imprimir <code>str(e)</code> , y esas vistas pasan de responder siempre 200 a 403/409/500	Era CWE-209 en una página visible sin sesión iniciada, y el 200 la hacía invisible para las alarmas. Nuevo <code>core/tests/test_error_token.py</code> (8 tests)
Se cierra el tercer frente, las APIs JSON: nuevo helper <code>json_error_inesperado</code> en <code>core/utils/json_responses.py:68</code> y nueve <code>except Exception</code> corregidos en <code>dias_especiales.py</code> , <code>calculo_automatico.py</code> , <code>turnos_mes.py</code> , <code>api_turno_jornada.py</code> y	<code>str(e)</code> de MySQL llegaba en el JSON: nombres de tabla y columna, y en el peor caso el endpoint de RDS. Seis de los ocho no dejaban traza (tres no registraban nada y tres usaban <code>logger.error</code> sin <code>exc_info</code>), así que el cambio además añade observabilidad. Se conserva un mensaje específico por endpoint en vez de un genérico único. Nuevo

<code>gestion_solicitudes.py</code>	<code>core/tests/test_json_error_inesperado.py</code> (16 tests). La novena era la peor: <code>MisTurnosPorMesView</code> publicaba el traceback entero a cualquier <code>is_staff</code> , y la primera versión del test —que comparaba línea a línea— no la detectaba. Ver § 9.5
Nuevo <code>core/tests/test_paginas_error.py</code> (19 tests) y <code>logs/</code> en <code>.gitignore</code>	Regresión sobre las fugas, el código de referencia y la autonomía de las plantillas. Los dos últimos se añadieron después, al descubrir que los <code>.catch()</code> de los formularios lanzarían <code>ReferenceError</code> si el stub de <code>CodigoReferencia</code> no estuviera
Nuevo <code>static/js/utils/codigo-referencia.js</code> , cargado en <code>templates/base.html:297</code> antes que jQuery, más el CSS de <code>static/css/base_custom.css:23-36</code> y ocho avisos de envío fallido en los seis <code>static/js/cambio-turno/solicitar_*.js</code>	Los formularios envían por <code>fetch</code> y no recargan: ante un 500 no pasaban por ninguna plantilla de error y el código se perdía pese a venir en <code>X-Request-ID</code> . Un shim sobre <code>window.fetch</code> lo recoge solo en respuestas con error y lo mantiene 60 s. Ver § 9.5

Seguridad de los enlaces de aprobación por correo

Cambio en el árbol de trabajo, todavía sin commit al redactar esta entrada (rama `fix/cambio-descanso-temporada`).

Cambio	Impacto
Nuevo <code>solicitudes/services/tokens_aprobacion.py</code> : fuente única de los tokens firmados, con <code>django.core.signing</code> sobre <code>SECRET_KEY</code> en lugar de una clave literal	Cierra B1 y B2 (§ 16.4). Sal propia por circuito: <code>solicitudes.aprobacion-email (:44)</code> y <code>permisos.aprobacion-email (:116)</code>
Se eliminan las seis copias duplicadas de la verificación: <code>services/email_service.py:528</code> , las cuatro de <code>views/aprobacion_email.py:140,199,251,303</code> y <code>permisos/services.py:34</code>	Un solo sitio que auditar. <code>permisos/</code> queda cubierto por primera vez
Nueva variable <code>APPROVAL_LINK_MAX_AGE_DAYS</code> (default 30) en <code>config/settings.py:262</code> y <code>.env.example:43</code>	Los enlaces caducan. Ver § 10
Nuevo <code>solicitudes/tests/test_tokens_aprobacion.py</code> (25 tests)	Firma, caducidad, sal, rol y suplantación
Condición operativa nueva: misma <code>'SECRET_KEY'</code> en	Añadida al checklist de despliegue (§ 13.3)

todas las instancias	
Generalizado como patrón 38 de PROTECTION_PATTERNS.md y razonado en el ADR 006 (./03-arquitectura/adr/006-tokens-firmados-para-aprobacion-por-correo.md)	El ADR recoge las alternativas descartadas (HMAC propio, tabla de tokens gastados, caché) y por qué la solución <i>stateless</i> es la que encaja con varias instancias tras un ALB
Corregidas 31 secuencias de mojibake en 5 archivos de solicitudes/views/	Cierra B6 (§ 16.4): el usuario ya no lee "Token inválido" en pantalla
Nuevo core/utils/error_token.py y plantilla error_token.html con el plazo y qué hacer	Cierra B7 (§ 16.4). Las 15 llamadas pasan por un punto único
Los seis print(...) de services/email_service.py pasan a logger.exception	Un print no lleva nivel ni traza: ese texto no llegaba útil a CloudWatch. Ahora sí, con la traza de la excepción

Temporada y cambio de descanso (rama fix/cambio-descanso-temporada)

Commit	Cambio	Impacto
40a7ed8	Tests de la reversión de las 5 opciones de temporada	Cubre P4 sobre CAMBIO DESCANSO
5bd991c	El select de compañero manda sobre la variable en memoria	Frontend: static/js/cambio-turno/solicitar_cambio_descanso.js
4bddab7	Los días de descanso fijados de temporada solo se cambian desde CAMBIO DESCANSO	Regla P3. Añade DescansoSemanaService.es_dia_descanso_temporada (turnos/services/descanso_semana_service.py:42) y el rechazo en doblada_strategy.py:189-197
9c429e3	El desplegable y la validación miran la misma fecha	Patrón 37; origen del filtro por día de pago en CoberturaCandidatosView
a7607d7	Doblada permanente: medir el choque con el compañero por FECHA	Corrige falsos negativos de disponibilidad
56eb793	Ceder un festivo	—

	entero cubre el día completo (AM+PM)	
--	--------------------------------------	--

Rendimiento y consistencia

Commit	Cambio
00558d8	CT permanente: precarga en lote de la matriz empleado × día
1fe8d6e y 1a5111e	Reconciliación: cerrar los días colaterales antes de re-aplicar (patrón 33)
1967790	Integridad: refrescar solo las fechas reconciliadas
0c3ccc8	Mis Turnos y D FDS: un día de fin de semana es DÍA COMPLETO, no una DOBLADA
a437c41	Registrar la hora real de la cancelación → campo <code>fecha_cancelacion</code> (<code>solicitudes/models.py:161-165</code>)
db728c9	Cambio descanso: ejercitar de verdad la regla del día en curso

Vocabularios y constantes

Commit	Cambio
6680164	Se crea <code>core/constants.py</code> como fuente única de los tres vocabularios
2442bc3	La base de datos valida <code>Turno.tipo_cambio</code> : choices + <code>CheckConstraint</code> (<code>turnos/models.py:90-94</code>)
52bec93	Los servicios que escriben turnos usan <code>TipoCambioTurno</code>
80b435e	Se extraen a constantes los literales repetidos de bajo riesgo
cdd8a2c	Se corrige la validación CT+doblada, que nunca disparaba
15056c2	Revert: se quita el bloqueo CT+doblada porque contradice la regla 18

Limpieza de código muerto

4f0798d (DiaEspecialCreateView), 4599f55 (alias /dashboard/ y API alternancia-finde), 99f0ac1 (PDHVisualizarListView), 1a18284 (tres vistas placeholder nunca implementadas), 19ca11e (CRUD manual de turnos, muerto desde 2025) y 02b1330 (.pyc versionados en core/).

Calendario y calidad

9d87547 introduce bloqueo optimista al guardar el año y elimina la ruta de baja; 4dd6cd6 hace que el sistema diga **por qué** un compañero no cubre una fecha; 339cdee añade la configuración de SonarQube, la cobertura y las correcciones del triaje.

18. Por confirmar

#	Afirmación pendiente	Dónde se buscó	Por qué no se pudo verificar
1	Alcance real de B5 (correo irretirable en cobertura con dos compañeros): si <code>SolicitudFactory.crear_solicitud</code> encola el <code>EmailOutbox</code> dentro de la misma <code>transaction.atomic()</code> , el rollback borra también la fila del outbox y el bug solo existe si el worker ya la reclamó	<code>solicitudes/services/solicitud_orchestrator.py:262-280</code> , <code>solicitudes/models.py:44-64</code>	Exige leer las seis estrategias y <code>EmailOutboxService</code> completo; excede el presupuesto de esta pasada
2	Lo mismo para <code>_procesar_doblada_permanente_multi</code> (<code>solicitud_orchestrator.py:366</code>)	Ídem	Ídem
3	Porcentaje de cobertura de tests actual	No se ejecutó <code>pytest --cov</code>	Requiere una base MySQL levantada
4	Número exacto de migraciones de <code>turnos</code> , <code>empleados</code> y <code>permisos</code>	<code>*/migrations/</code>	Solo se contaron las de <code>solicitudes</code> (35 archivos). La § 1.3 cita 33/15/8/8 de una pasada anterior; el dato de <code>solicitudes</code> ya no coincide y conviene

			recontar los cuatro
5	Contenido de <code>instructivos/*.docx</code> y <code>*.mwb</code> (reglas dictadas por el negocio)	<code>instructivos/</code> , raíz del repositorio	Formato binario, no contrastable con el código
6	Contenido de <code>04-guias/manuales/*.docx</code> ("requisito doblada", "Proceso completo doblada", "estructura sql") y de <code>PLAN_CORREO_TRANSACCIONAL_Y_LATENCIA.docx</code>	<code>docs/04-guias/manuales/</code> , <code>docs/05-referencia/deployment/</code>	Ídem
7	Si existe integración con Google Apps Script	Búsqueda en <code>AppTurnosExplora/</code>	No se encontró referencia en el código; puede vivir fuera del repositorio
8	Qué vistas concretas de <code>turnos/</code> , <code>empleados/</code> y <code>permisos/</code> exigen rol supervisor	<code>core/mixins.py</code> , <code>*/urls.py</code>	Se verificó el mecanismo (<code>AdminRequiredMixin</code> , <code>SupervisorApiRequiredMixin</code>), no la lista vista por vista
9	Si la CSP report-only ha generado violaciones desde su publicación	<code>config/settings.py:399-418</code>	Requiere leer los informes del navegador en un entorno desplegado
10	Contenido exacto del <code>Dockerfile</code> (imagen base, usuario, <code>HEALTHCHECK</code> , <code>ENTRYPOINT</code>)	<code>AppTurnosExplora/Dockerfile</code>	Solo se verificó la versión de Python; el resto se cita desde documentación previa
11	Frecuencia real del cron de <code>procesar_email_outbox</code> en producción	<code>MANUAL_OUTBOX_CORREOS.md</code> (<code>./05-referencia/deployment/MANUAL_OUTBOX_CORREOS.md</code>)	No está en el código: es configuración de infraestructura
12	Si la sección "deuda conocida pendiente de decisión" de <code>PROTECTION_PATTERNS.md:1731</code> sigue vigente	<code>PROTECTION_PATTERNS.md</code>	Se leyó el índice, no el contenido completo de esa sección
13	Si <code>CierreSolicitudesConfig</code>	<code>solicitudes/models.py:802</code>	El valor por defecto es <code>False</code> ; el estado

	.habilitado está activo en el entorno real		real es un dato de producción
~~1 4~~	~~Si la SECRET_KEY procede de una sola entrada de Secrets Manager/SSM o de un valor por tarea~~	RESUELTO — sí procede de una sola entrada; ver § 9.2	La auditoría lo resolvió con fuentes del propio repositorio: el checklist de Fargate crea swalp/SECRET_KEY como secreto único y lo inyecta vía secrets: del task definition, y la ruta EC2 es de instancia única. Además la aplicación no puede generar una clave propia

Fin del manual técnico. Fuente de verdad: `AppTurnosExplora/docs/manual\_tecnico.md`. El PDF de `docs/pdf/Manual\_Tecnico.pdf` se genera desde este archivo y nunca se edita a mano.